BIG DATA / REPRODUCIBLE LABORATORY

Big Data Lab Guide

Streaming affinity operations, activity analytics and an S3-compatible data lake.

Document control	Value
Document ID	MSC-BD-001
Version / released	1.5.23 / 2026-10-28
Owner	myscoutee-bigdata
Audience	MyScoutee developers learning data pipelines and evaluating integration
Scope	Runnable core and six isolated stations covering the original 12 technologies
Qualification	Lab behavior only; no production backend or telemetry feed is connected
Source of truth	The backend owns affinity calculation, task processing and product state

This guide distinguishes the running laboratory from proposed product integration. A replacement is a decision to evaluate after measured scale, correctness and operational evidence; it is not implied by a working local demo.

Contents and learning path

Section	Page
1. Runnable architecture and technology roles	3
2. MyScoutee use cases and current owners	4
3. Flink and affinity: useful boundary	5
4. Integration contract and data governance	6
5. Replacement decisions	7
6. Hands-on runbook and exercises	8
7. Scope, limits and references	9
8. Original 12 technologies: roles and exercises	10-12
Hadoop and MapReduce: terms and data path	11
Hive: guided first run	13
Hive: SQL, metadata and files	14
MyScoutee integration map: activity and affinity	15-16
9. Staged Docker classroom and resource policy	17
10. Observe each isolated station	18
HBase: core concepts and write path	19
HBase: guided first run	20
Cassandra: concepts and write path	21
Cassandra: guided first run	22
Cassandra: cluster model and MyScoutee fit	23
Airflow: guided first run	24
Airflow: comparisons and optimized roles	25
Airflow: MyScoutee adoption examples	26
NiFi: guided first run	27
11. Observe the Kafka, Spark and Flink core	28
Kafka: concepts and guided observation	29-31
Kafka vs work queues; groups; adoption design	32-34
ClickHouse: guided visit, storage and choices	35-38
Spark, Parquet and SeaweedFS: concepts and visit	39-44
Flink: execution, affinity UI and time semantics	45-47
Avro + Schema Registry: contract, lab and schema	48-50

Learning sequence

Start with a small activity stream. Read the live ClickHouse projection, then recompute its historical view with Spark and inspect the Parquet objects in SeaweedFS. Start the Flink profile, publish synthetic affinity task transitions, and inspect windows and checkpoints. Finally, compare the local result with the backend's actual responsibilities before proposing an integration.

Station	Question to answer
Kafka	What happens when a consumer stops and later resumes at an offset?
ClickHouse	How do a raw event table and a materialized view support fast counts?
Spark + Parquet	How can historical data be deduplicated and partitioned for later scans?
SeaweedFS S3	What changes when the batch output is an object rather than a local file?
Flink	How do windows, checkpoints and replay affect affinity task telemetry?
Avro + Registry	How can a new optional field be added while old records still decode?

1. Runnable architecture

```
synthetic activity producer -> Kafka activity.v1
                            |-> ClickHouse raw events -> dashboard
                            +-> Spark batch -> Parquet -> SeaweedFS S3

synthetic affinity producer -> Kafka affinity.tasks.v1
                            -> Flink 10 s processing-time windows
                            -> Kafka affinity.rollups.v1
                            -> ClickHouse rollups -> dashboard
```

Component	Implemented role	Boundary
Kafka	Three topics; one KRaft broker; seven-day local retention.	Transport and replay, not durable product state.
ClickHouse	Kafka Engine ingestion, materialized views and local query API.	Analytical projection, not transactional writes.
Spark	On-demand Kafka history scan, event-ID deduplication and daily Parquet aggregates.	Batch snapshot; rerun after new data.
SeaweedFS mini	Single-node S3-compatible bucket for Parquet output and readback.	Lab object store, not an HA lake.
Flink	Task transition rollups, checkpointed state and Kafka sink.	Operational telemetry; no affinity scores.
Dashboard	Read-only activity and affinity views.	Separate from the product admin UI.

The default lab.sh up path starts Kafka, ClickHouse, SeaweedFS and the dashboard. Spark runs only for batch; Flink runs only for stream-up. Six independent stations start through stations.sh, one at a time. Published ports bind to 127.0.0.1. Docker volumes retain data across down; reset deletes them.

The topology is deliberately single-node. It teaches component roles and failure modes without claiming production throughput, high availability or exactly-once results.

2. MyScoutee use cases

Use case	Existing owner and signal	What this lab adds
Activity funnel	Backend telemetry includes profile.registered, profile.onboarding and event.member.joined; event creation and removal also exist.	Unique-actor counts and city/mode trends over historical windows.
Affinity queue health	AffinityRecomputeTaskService stores Mongo tasks; change stream wakes the worker; metrics track queue and runtime.	Independent pending/running/done/failed and target-type windows in Flink.
Historical replay	Product APIs and admin projections answer current application questions.	Kafka retention and Spark batch show replay, deduplication and one overwritten Parquet output.
Exploration	Existing Elastic repository owns search/observability experiments.	ClickHouse answers numeric event questions; object storage holds exportable history.

Activity analytics

The generator emits MyScoutee-shaped event keys, actor IDs, event IDs, modes (Casual/Mingle), cities and timestamps. The join funnel is meaningful because synthetic joins follow onboarding. City and mode are simulated dimensions: a live export must confirm that the backend really supplies permitted values before using them in a report.

Affinity operations

Real recompute targets are USER, EVENT, ASSET and CONFIG. Reasons include user, event, membership, asset/request, admin-status and configuration changes. The task lifecycle is pending, running, done or failed. The lab models these transitions, then Flink reports counts and wait-time summaries by target. The backend's Mongo task queue and worker remain authoritative.

Decision point

The best first integration candidate is an exported, minimized task telemetry stream for operations analytics. A live activity stream is a separate candidate for cohort reporting. Neither requires moving the affinity score algorithm.

3. Flink and affinity

What the backend calculates

AffinityCalculationService combines profile facts, distances, ratings and configuration. AffinityRecomputeWorkerService performs durable queued work. Replacing either requires full input semantics, update ordering, retry rules and score parity. The lab has none of those inputs; its Flink SQL computes task-state telemetry only.

Flink is useful now	Flink could become a score engine only if...
Aggregate task starts/completions/failures by target and time window; detect rising queue delay; power an operations panel.	All score inputs are versioned events with reproducible ordering; replay equals the canonical backend result; privacy, deletes, backfills and rollback are designed.
Checkpoint operator state and observe recovery after a task manager restart.	A durable handoff replaces the Mongo queue safely, with idempotent writes and measured latency/cost improvement.

Window semantics

The example uses 10-second processing-time windows. Arrival delays and restarts can move a transition into another window. Kafka sink delivery is at least once; the ClickHouse rollup table merges duplicate window keys, but a new job replaying earlier messages can still produce a different processing-time window and extra counts. Do not use this output for billing, compliance or exact product counters.

A production candidate would use event timestamps and watermarks, stable task/event IDs, deduplication state, transactional or idempotent sinks, replay tests and explicit late-event handling. It must also compare Flink metrics against the backend's AffinityRecomputeMetricsService.

```
./lab.sh stream-up
./lab.sh affinity-seed --tasks 1000 --seed 42
./lab.sh query "SELECT target_type, sum(pending), sum(done),
  sum(failed) FROM affinity_rollups FINAL GROUP BY target_type"
```

4. Integration contract

Narrow route from product to lab

```
backend-owned durable task/event -> reviewed export adapter
  -> minimized, versioned event -> Kafka -> lab consumers
  -> compare with backend metrics -> only then consider production
```

Field / rule	Proposed contract
Identity	Stable event_id and task_id for deduplication; no profile payload or raw target ID in analytical topics.
Envelope	schema_version, occurred_at UTC, event_key, source, export time; document ordering key and retry behavior.
Affinity fields	target_type, status, reason, coarse group, queue wait and runtime milliseconds when available.
Activity fields	Only the cohort dimensions needed for an approved report; actor pseudonymization and deletion policy.
Operations	Retention, access controls, backfill, schema evolution, dead-letter handling and consumer lag alerts.

Export ownership

Prefer a backend-owned post-commit publisher or outbox, so a committed task or activity event has a durable, replayable export. The backend should own the canonical mapping from its domain model. The current Redis admin-telemetry stream is backend-owned with its own retention and worker consumption: attaching another production consumer directly is not a safe integration shortcut.

Validation gates

Before connecting real data, compare a defined sample against the backend's task and admin counters; prove recovery after duplicate sends, Kafka unavailability and schema changes; review personal-data minimization and retention. The existing sandbox has no credentials or network path to production.

5. Replacement decisions

Existing part	Candidate relationship	Gate before replacement
Mongo affinity task queue + worker	Keep. Flink can consume exported transitions for telemetry.	Score parity, durable retries, input ordering and latency evidence before any architectural change.
Redis admin telemetry stream	Keep as the backend's operational stream; export approved data through its owner.	Contract and retention ownership; no competing direct consumer.
Product admin statistics	Compare with ClickHouse for broad historical analytics; keep current API until validated.	Agreed metric definitions, data quality, latency and query cost.
Elastic search/observability	Complement with analytical SQL; ClickHouse is a candidate for numeric aggregations.	Test real search, log and dashboard workloads separately.
GitLab CI schedules	Keep for builds and releases. A future orchestrator could schedule data-only backfills.	Enough recurring dependencies and operational value to justify Airflow.
Mongo operational storage	Keep for application writes and task lifecycle.	Analytical stores are not drop-in transactional replacements.

A sensible adoption order

1. Measure the current questions and volumes. 2. Export a minimized event contract. 3. Run the lab in parallel against a safe dataset. 4. Compare results and failure recovery. 5. Promote a specific analytics view only when its owner and operational needs are clear. Affinity calculation replacement is a separate, much higher-risk project.

Technology is justified by workload

At small local volumes, the stack is educational overhead. Kafka/Flink help when high-rate, independently consumed events and continuous windows are real requirements. Spark and an object lake help when repeated history/backfills exceed a simple query. ClickHouse helps when many large aggregate queries must be fast. Measure before committing to any of these in production.

6. Hands-on runbook

```
./lab.sh up
./lab.sh seed --users 5000 --days 30 --seed 42
./lab.sh batch
./lab.sh stream-up
./lab.sh affinity-seed --tasks 1000 --seed 42
./lab.sh status
```

Endpoint / command	What to inspect
http://127.0.0.1:8094	Read-only activity and affinity dashboard.
http://127.0.0.1:8081	Flink job, task managers and checkpoint history after stream-up.
http://127.0.0.1:18080	Spark History Server after history-up; completed job, stage and task details.
http://127.0.0.1:8888	SeaweedFS Filer UI; browse bucket prefixes and Parquet objects.
http://127.0.0.1:8333	S3-compatible endpoint; bucket myscoutee-lake, lab-only credentials in Compose.
./lab.sh query "SELECT count() FROM activity_events"	ClickHouse raw ingestion count.
./lab.sh down / reset	Stop while retaining data / delete the lab's volumes.

Inspect the Parquet lake

```
AWS_ACCESS_KEY_ID=lab AWS_SECRET_ACCESS_KEY=lab-only-secret \
  aws --endpoint-url http://127.0.0.1:8333 s3 ls \
  s3://myscoutee-lake/daily_activity/ --recursive
```

Three experiments

Consumer lag: stop ClickHouse, emit more activity, restart and observe the catch-up. History: run batch, emit a second seed, run batch again and compare the S3 day partitions and row count. Streaming: inspect Flink checkpoints, emit another affinity batch and compare pending, done and failed rollups.

Verification

The synthetic generator's unit tests check event shape and reproducibility. A full smoke run should confirm Kafka ingestion in ClickHouse, Spark write and readback from S3, a running Flink job with successful checkpoints, and visible affinity rollups. These are local lab checks, not production acceptance tests.

7. Scope and limits

Technology	Where it fits / next learning station
Hadoop / HDFS / YARN / MapReduce	One isolated cluster for storage, scheduling and a join-count batch job. Compare with the existing S3 + Spark path.
Hive / HBase / Cassandra	Separate SQL warehouse and two row-store stations. The same synthetic activity and affinity fixtures make comparisons reproducible.
Airflow / NiFi	Orchestrate a synthetic cohort report; visually validate and route synthetic JSON. Run as separate stations.
Elastic Stack	Search and observability have a separate MyScoutee repo; use this lab for event aggregates and history.

Known limits

One broker, one object-store node, no TLS, fixed development credentials and local loopback ports. Kafka retains messages for seven days; ClickHouse and SeaweedFS volumes persist until reset. The first Spark batch downloads a large Hadoop S3 dependency and needs several GB of free working space. Synthetic actor and city data must never be mistaken for measured MyScoutee behavior. Processing-time Flink rollups are illustrative and replay-sensitive.

Sources and implementation

Project files: lab.sh, stations.sh, stacks/core/compose.yaml, stacks/core/sql/, stacks/core/flink/affinity.sql, stacks/core/spark_batch.py, producer/, stacks/core/dashboard/ and README.md. Existing MyScoutee owners: backend AffinityRecomputeTaskService, AffinityRecomputeWorkerService, AffinityRecomputeMetricsService, AffinityCalculationService and AdminTelemetryService.

Official technology references: kafka.apache.org/41/getting-started/docker/; clickhouse.com/docs/integrations/connectors/data-ingestion/kafka/kafka-table-engine; spark.apache.org/docs/4.0.4/streaming/structured-streaming-kafka-integration.html; nightlies.apache.org/flink/flink-docs-release-2.2/docs/deployment/resource-providers/standalone/docker/; hadoop.apache.org/docs/r3.4.2/hadoop-project-dist/hadoop-common/SingleCluster.html; github.com/seaweedfs/seaweedfs#quick-start.

8. The original 12: foundations

The source list has 12 names. Hadoop, HDFS, YARN and MapReduce share one learning station. Each station has a separate Compose project, deterministic fixture, exercise and stop command; run one at a time on a laptop.

Technology / role	MyScoutee-shaped exercise	Repository state
Hadoop - platform	Follow one synthetic activity export from upload through a scheduled batch job; identify each daemon.	./stations.sh hadoop up/run/down
HDFS - filesystem	Store 71 join tokens; inspect blocks, replication factor one and NameNode UI. Compare with SeaweedFS objects.	Same Hadoop station; NameNode on 9871.
YARN - resources	Inspect the ResourceManager and one NodeManager while the MapReduce application runs.	Same station; ResourceManager on 8089.
MapReduce - batch model	Count synthetic joins by city and mode: 55 Bratislava_Casual and 16 Praha_Casual.	WordCount example runs through YARN.
Spark - batch engine	Run the existing Kafka-history scan, deduplicate event IDs and write/read partitioned Parquet for daily activity aggregates.	Available via ./lab.sh batch.
Kafka - event log	Publish activity and affinity task signals, stop a consumer, inspect offsets and lag, then replay retained events.	Available; one KRaft broker, three topics.

Learning distinction

HDFS stores files; YARN schedules work; MapReduce is one processing model; Hadoop bundles the ecosystem. Spark can run locally, on YARN, or elsewhere. The current Spark job does not exercise HDFS, YARN or MapReduce.

How the storage and compute sides connect



A MapReduce application reads and writes HDFS files while YARN allocates resources for its tasks. The ApplicationMaster coordinates that application's tasks and requests containers from the ResourceManager. A YARN container is a resource allocation and execution unit; it is not one of this lab's Docker containers.

This lab exposes the NameNode on 9871 and the ResourceManager on 8089. Hadoop commonly uses 9870/8088; this Compose project does not start a JobHistory UI on 19888. It has one DataNode and one block replica, so it does not demonstrate node-failure survival.

References: HDFS Architecture and Apache Hadoop YARN.

Hadoop and MapReduce: a first mental model

The Hadoop station combines storage (HDFS), resource scheduling (YARN) and a batch calculation (MapReduce). Its 71 records are synthetic MyScoutee-shaped event joins, one city/mode token per line; this small input teaches the path, not a scale benchmark.



YARN schedules the map and reduce tasks; HDFS holds the input and output files.

Core terms on the running station

Term	Meaning and what you can see
HDFS / NameNode	HDFS is the distributed file system. The NameNode keeps the file tree and block locations; its UI shows files and DataNode health.
DataNode / block / replica	A DataNode stores file blocks (data bytes). A replica is another copy of a block. This one-DataNode lab uses replication factor 1, so it has no redundant copy.
YARN / ResourceManager	YARN manages compute resources. Its ResourceManager accepts the MapReduce application and assigns resources to run it.
NodeManager / container	The NodeManager manages work on its node. A YARN container allocates memory and CPU resources for an application task; it is not a Docker container.
ApplicationMaster	One per application: it requests containers from the ResourceManager and coordinates the map and reduce tasks on NodeManagers.
Application / job	The submitted WordCount job is one YARN application. Its map and reduce tasks have progress and final status in the ResourceManager UI.

What the MapReduce job does

Stage	This lab's example
Input / split	HDFS stores 71 lines in /myscoutee/joins/input.txt. One logical input split creates one map task here.
Map: key-value pairs	WordCount reads each line and emits (city_mode, 1), such as (Bratislava_Casual, 1).
Combiner	An optional local sum reduces transfer. In this run, 71 map pairs became 2 combiner outputs; correctness cannot depend on this optimization.
Shuffle / sort	The framework transfers and groups intermediate values by key, so all counts for the same city/mode reach the reducer together.
Reduce / output	The reducer sums each group and writes part-r-00000 in HDFS: Bratislava_Casual 55 and Praha_Casual 16. _SUCCESS marks completion.

On the NameNode home screen, inspect one live DataNode, then browse /myscoutee/joins for input.txt and result/part-r-00000. On the ResourceManager home screen, Applications shows word count as FINISHED / SUCCEEDED; Nodes shows one running NodeManager. The two counts sum to the 71 input records.

Reference: Apache Hadoop 3.4.2 MapReduce Tutorial.

8. The original 12: data systems

Technology / role	MyScoutee-shaped exercise	Repository state
Flink - stateful stream	Watch 10-second affinity task-state windows, checkpoints and replay effects. Compare the output with backend queue metrics.	Available via ./lab.sh stream-up.
Hive - SQL warehouse	Load the synthetic activity CSV; query registrations, onboarding, joins and city/mode counts.	./stations.sh hive up/run/down
HBase - sparse row store	Put and scan 40 latest affinity task states by target-type/task row key.	./stations.sh hbase up/run/down
Cassandra - wide-column model	Query the same 40 task states by target type and day with a partition-aware CQL table.	./stations.sh cassandra up/run/down
Airflow - orchestration	Run extract -> validate -> publish on synthetic registration, onboarding and join counts.	./stations.sh airflow up/run/down
NiFi - visual dataflow	Route valid and invalid synthetic JSON into accepted/rejected files; inspect provenance.	./stations.sh nifi up/run/down

Useful comparisons

Hive and ClickHouse both accept SQL, but Hive is the file/lake warehouse lesson and ClickHouse is the existing fast analytical serving layer. HBase and Cassandra teach different partition and key design; neither is a direct replacement for Mongo's product writes or the durable affinity task lifecycle. Airflow schedules jobs; NiFi routes data; Kafka transports events; Flink processes them continuously.

Official starting points: hive.apache.org/docs/latest/admin/setting-up-hive-with-docker/; hbase.apache.org/docs/getting-started/; cassandra.apache.org/doc/latest/cassandra/getting-started/cassandra-quickstart.html; airflow.apache.org/docs/apache-airflow/3.1.7/howto/docker-compose/index.html; nifi.apache.org/nifi-docs/getting-started.html.

Hive: terms behind the next station

Term	Meaning in this lab
Hive / HiveQL	Hive adds SQL-style analysis over stored files. HiveQL is the query language; the lab uses GROUP BY to count event types and joins by city/mode.
Database / table	myscoutee_lab is the namespace; activity_events is a six-column table over text rows from the synthetic CSV.
Metastore / warehouse	The metastore records schema and file locations; the warehouse holds table data. This isolated image uses embedded Derby metadata and its own local warehouse volume.
HiveServer2 / Beeline	HiveServer2 accepts JDBC queries on port 10000 and has a status UI on 10002. Beeline is the command-line client that sends SQL and prints result rows.
LOAD DATA LOCAL / OVERWRITE	LOCAL reads activity.csv from the Hive container; OVERWRITE replaces the previous table contents. Loading copies data without validating the event schema.
Tez / query plan	Hive compiles the GROUP BY queries into Tez work; Beeline shows map/reducer-style vertices. This station is separate from the Hadoop Compose project.

In the HiveServer2 Home page, Active Sessions and Open Queries can be 0 after Beeline exits; Last Max 25 Closed Queries should list the completed statements. The actual counts appear in the Beeline output: 120 registered, 103 onboarded, 71 joined, then 55 Bratislava and 16 Praha joins. These are independent event counts, not yet a person-linked cohort.

References: HiveServer2 Overview and Hive DML manual.

Hive: guided first run

Purpose and data path

This station lets you query a synthetic MyScoutee activity export with SQL. It uses its own local warehouse and embedded metastore; it does not read the separate Hadoop station's HDFS or connect to the backend.



The metastore holds schema/location; the warehouse holds data; HiveServer2 serves SQL.

1. Read the HiveServer2 Home page

Open http://127.0.0.1:10002 after ./stations.sh hive up and ./stations.sh hive run. No login is configured in this isolated lab. The page monitors the SQL service; it is not a SQL editor.

Area	What it means after this lab run
Active Sessions	Connected clients. Usually 0 because the Beeline command has exited.
Open Queries	Statements still executing. Usually 0 after the two GROUP BY queries finish.
Last Max 25 Closed Queries	Six completed statements: CREATE DATABASE, USE, CREATE TABLE, LOAD DATA, and two SELECTs. anonymous is the lab client identity; tez is the execution engine; FINISHED is the statement state.
Software Attributes	Hive 4.0.0 and server start details, not the activity-event counts.

2. Open a query's Drilldown

Choose the SELECT city, mode query in Closed Queries and open Drilldown. Stages shows Stage-1:MAPRED with Success; Performance Logging shows timing for compilation and execution. Query Plan says future plans need hive.server2.webui.explain.output=true. An old Operation Log may show FileNotFoundException because that log was deleted after completion; check the FINISHED state and Beeline result instead.

3. Read the actual result

```
./stations.sh hive run
# script: stacks/hive/activity.hql
SELECT event_key, COUNT(*) FROM activity_events GROUP BY event_key;
```

Beeline prints 120 registrations, 103 onboarding events and 71 joins. Its second query filters event.member.joined and groups by city and mode: Bratislava/Casual 55, Praha/Casual 16. On the Home page, Local logs, Metrics Dump, Hive Configuration, Stack Trace and LLAP Daemons are server diagnostics; none displays the SELECT result rows.

MyScoutee integration boundary

A future historical participation cohort would need a reviewed, minimal export, stable pseudonymous actor key, event times, day partitions and defined 7/30-day windows. This lab only counts event records; it does not link each person's registration to onboarding and first join. The backend remains the product source of truth.

Hive: SQL, metadata and files

Follow one query

```
Beeline sends SELECT -> HiveServer2 manages the session
  -> compiler/optimizer reads schema and location from Metastore
  -> Tez scans warehouse files, groups rows, returns results
  -> HiveServer2 sends result rows back to Beeline
```

The metastore is a catalog of tables, columns, partitions and file locations; it does not hold the event rows. The query processor plans the work, and a SerDe interprets stored bytes as fields. The current station runs Tez and its own local warehouse. Its separate Hadoop station is not supplying HDFS or YARN to this query.

Core terms

Term	Meaning	This lab / MyScoutee consequence
Driver, compiler, optimizer	Manage the query lifecycle, interpret HiveQL and choose an execution plan.	Drilldown shows stage and timing; the SQL result is in Beeline.
SerDe	Serializer/deserializer maps stored bytes to table fields.	The current table parses comma-delimited text; loading it does not validate the event contract.
Managed table	Hive owns table data and metadata; DROP TABLE can remove both.	activity_events is a managed text table in the isolated warehouse.
External table	Hive catalogs files managed outside Hive; dropping the table normally leaves those files.	A future approved Parquet lake could be queried this way after connector, schema and catalog setup.
Partition / pruning	A partition is a logical slice such as event_day; a date filter can avoid scanning unrelated slices.	This CSV table is not partitioned. A partition is not an HDFS block.
Parquet / ORC	Column-oriented formats can read selected columns more efficiently than text scans.	Spark writes Parquet in SeaweedFS, but this Hive station does not read it.

A concrete MyScoutee question

For an approved export, a future Hive external table could let an analyst inspect one event-day partition of registrations, onboarding and first joins before publishing a 7-day cohort. That requires stable pseudonymous actor keys, a complete export, partition registration and agreed late-event rules. The current GROUP BY outputs are event counts, so they cannot answer the person-linked cohort question.

References: HiveServer2, table ownership, partition pruning and SerDe.

MyScoutee: participation analytics path

Product question

Of the people who registered on a day, how many completed onboarding and joined their first event within 7 or 30 days? The backend's AdminStatsService already holds operational activity counters. A historical cohort requires linking approved event records by one stable pseudonymous actor key, applying an agreed time window and handling late events and deletions.

```
backend-owned approved export -> NiFi validation -> Kafka event log
  -> Spark cohort build -> Parquet in SeaweedFS
  -> Hive exploratory SQL / ClickHouse published aggregates
Airflow schedules and checks each daily batch step
```

Technology	Concrete MyScoutee role	Lab evidence / integration gap
NiFi	Route a backend-owned daily activity export by schema and required fields; quarantine invalid records.	Current flow only checks event_key in two JSON files; no backend export or Kafka sink.
Kafka	Retain approved profile.registered, profile.onboarding and event.member.joined events for independent consumers and replay.	Synthetic activity.v1 topic; a durable backend outbox and versioned envelope would be needed.
Spark + SeaweedFS	Deduplicate event IDs; compute first-join cohorts; store day-partitioned Parquet snapshots for reruns.	The current Spark batch makes daily event aggregates, not person-linked cohorts; SeaweedFS is a single-node lab store.
Hive	Use SQL over reviewed historical files to check registration, onboarding and first-join definitions before publication.	Hive currently counts a synthetic CSV in its own warehouse; reading SeaweedFS would need an object-store connector and external table.
ClickHouse	Serve approved aggregate cohort trends by date, city and mode to an analytics view after reconciliation.	Current Kafka materialized view counts synthetic events; no product admin view uses it.
Airflow	Schedule export readiness, validation, cohort calculation and publish; expose retries and failed partitions.	Current three-task DAG reads local JSONL and writes one JSON report; it has no schedule or external jobs.

Hadoop learning branch

For a future large, file-based export already hosted on Hadoop, HDFS could hold partitions, YARN could allocate batch work and MapReduce could produce audit counts. The separate station demonstrates this with 71 join tokens and a 55/16 city/mode count. Its WordCount does not calculate person-linked cohorts; the current MyScoutee volume and lab provide no reason to replace the Spark/object-storage route.

Example investigation: if the published 7-day first-join rate falls for a city, check Airflow's partition run, inspect the approved source rows in Hive, recompute the cohort with Spark and publish only reconciled aggregate counts to ClickHouse. The current lab cannot perform this end-to-end investigation.

MyScoutee: affinity operations path

Product question

After user, event, membership, asset or configuration changes, are recomputations waiting longer, failing more often or concentrated in one target type? AffinityRecomputeTaskService owns the Mongo task lifecycle; AffinityRecomputeWorkerService executes it; AffinityRecomputeMetricsService and Admin Monitoring expose current operational signals. The score stays in AffinityCalculationService.

```
committed task transition -> backend-owned minimal export
  -> Kafka affinity.tasks.v1 -> Flink windowed telemetry
  -> Kafka affinity.rollups.v1 -> ClickHouse history
  -> compare with backend metrics and Admin Monitoring
```

Technology	Concrete MyScoutee role	Lab evidence / integration gap
Kafka	Fan out versioned task transitions to analytics without making the broker the authoritative task queue.	Synthetic task topic; no real task export and no score input stream.
Flink	Watch pending/running/done/failed rates and queue wait by USER, EVENT, ASSET and CONFIG target.	Ten-second processing-time windows and checkpoints; replay may change window counts.
ClickHouse	Keep fast historical rollups for questions such as failure rate by day and target type.	Current rollup table is a lab projection; reconcile against backend metrics before use.
Airflow	For a daily audit, compare exported transition totals with backend metrics and publish a corrected historical partition.	An adoption candidate; the present DAG only processes activity fixture data.
NiFi	If approved exports arrive as files, check and quarantine malformed envelopes before downstream processing.	Optional intake boundary; it does not schedule recompute tasks or calculate scores.

What would make the integration real

Step	Required evidence
1. Export	Backend owner emits post-commit, versioned events with stable task/event IDs, UTC occurrence time, target type and status; no raw profile payload.
2. Verify	Define duplicate, late-event, retention and deletion handling; compare a safe sample with AffinityRecomputeMetricsService and AdminStatsService.
3. Decide	Measure whether a separate historical/streaming answer is needed beyond Admin Monitoring and Jobs. Keep Mongo tasks and backend workers authoritative.

Current status: both paths above are proposed integrations. The Docker lab runs on synthetic fixtures. No station is wired to the live MyScoutee backend.

Example investigation: if EVENT tasks become pending after many event membership changes, compare the backend's queue and worker metrics with Flink's target-type windows. A disagreement may reflect late or replayed telemetry; inspect event IDs and window time before interpreting it as a product failure.

9. Staged Docker classroom

Target layout

Station	Composition and acceptance evidence
A - stream and batch	Kafka, Flink, Spark, ClickHouse and SeaweedFS. Show a checkpoint, Kafka offset and Parquet object.
B - Hadoop fundamentals	HDFS, YARN and MapReduce. Show NameNode/ResourceManager UIs, one DataNode/NodeManager and the 55/16 join count.
C - warehouse and row store	Hive queries the activity CSV; standalone HBase stores the final affinity states. Compare SQL scans and row-key access.
D - Cassandra	One isolated node. Show a CQL keyspace, table and partition-aware affinity task query.
E - NiFi	Five-processor JSON validation flow. Show accepted/rejected files, queues and data provenance.
F - Airflow	Three-task cohort DAG. Show dependencies, a successful test run and the JSON report.

Resource policy

The host has 15 GiB RAM. Start one station at a time, and stop the core lab before memory-heavy stations. Images are version-pinned, UIs bind to 127.0.0.1, services have memory caps, and independent Compose projects retain their own volumes. Disk cleanup freed enough working space for the image pulls without removing Elastic data or the .codex conversations. Inspect free disk before rebuilding images or running all jobs.

Implementation order and decision gate

Use ./stations.sh STATION up, run and down; see stacks/README.md and the next two pages for endpoints, sample output and details. The default fixture contains 306 activity events and 40 affinity task lifecycles. Hadoop and Hive must agree on the 55/16 join breakdown; HBase and Cassandra must each hold 40 final task states. This is a reproducible classroom, with no live backend connection or production replacement implied.

10. Observe the isolated stations

Run one station at a time

```
cd /home/raxim/workspace/myscoutee-bigdata
python3 producer/export.py --users 120 --tasks 40 --seed 42
./stations.sh STATION up
./stations.sh STATION run
./stations.sh STATION down
```

Replace STATION with a name below. Keep the station running while inspecting its UI. The terminal output is part of the exercise: some systems have no query UI in this lab. The expected numbers below come from the default synthetic seed, not from MyScoutee production.

Station	Open / inspect	Evidence after run
hadoop	NameNode: http://127.0.0.1:9871 - one live DataNode and the HDFS input. ResourceManager: http://127.0.0.1:8089 - one completed application and a running NodeManager.	The terminal prints Bratislava_Casual 55 and Praha_Casual 16. Their sum is 71 joins.
hive	HiveServer2 status: http://127.0.0.1:10002. Beeline output from run is the actual query view.	Counts: 120 registered, 103 onboarded, 71 joined; city/mode joins 55 and 16.
hbase	HBase Master: http://127.0.0.1:16011. Inspect the affinity_tasks table and server status; read the shell scan printed by run.	The shell prints 40 rows, followed by five sample row keys and status/group cells.
cassandra	No web UI is provided. Read the CQL output from run; use cqlsh inside the container for further queries.	The table has 40 final task states; the example queries five rows in the USER/day partition.
airflow	UI: http://127.0.0.1:8096. Open myscoutee_activity_cohort and its run graph: extract -> validate -> publish.	All three tasks succeed. The terminal prints a JSON report with 120/103/71 and 55/16 counts.
nifi	UI: https://localhost:8444/nifi. Trace GetFile -> EvaluateJsonPath -> RouteOnAttribute -> PutFile; inspect Data Provenance.	build/stations/nifi-output/valid/valid.json has event_key; rejected/invalid.json lacks it.

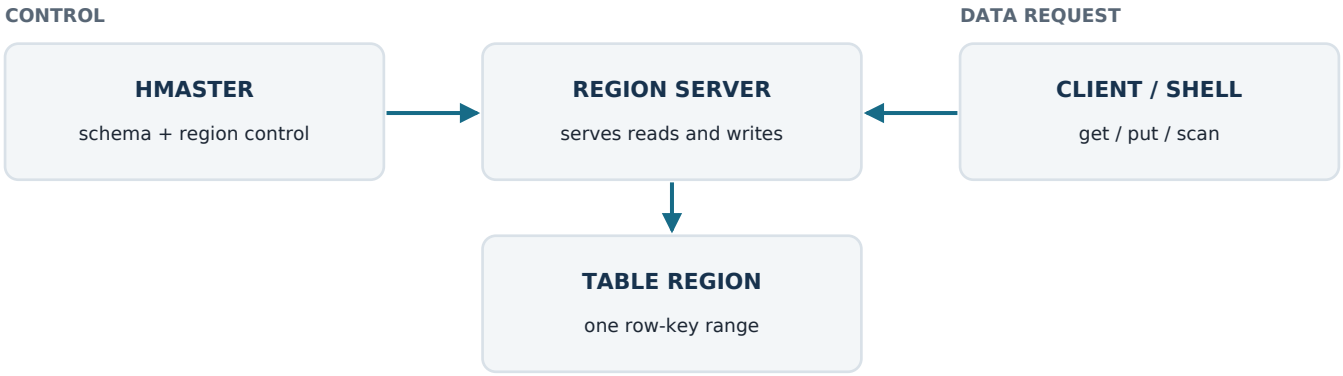
The next two pages explain HBase and its Master UI. Keep its container running while you inspect the table; Cassandra follows afterward.

NiFi uses labuser / lab-only-password-123. Its local HTTPS certificate may trigger a browser warning. GetFile consumes the two input files; rerun producer/export.py before repeating the NiFi exercise. A fresh NiFi container needs stations.sh nifi run again to recreate its canvas.

HBase: core concepts and write path

A sparse, versioned map

Think of an HBase cell as (row key, column family, qualifier, version timestamp) -> value. Row keys are sorted as bytes. A family is declared with the table, while a row may omit qualifiers it does not need. This lab stores the same two qualifiers for every task, so it does not exercise sparse optional fields.



Part	Responsibility	What this lab shows
HMaster	Manages table metadata and region assignment; monitors RegionServers.	The Master UI lists affinity_tasks and one Normal RegionServer.
RegionServer	Serves get, put and scan for its assigned key ranges.	One RegionServer hosts the affinity_tasks region and hbase:meta.
Region / hbase:meta	A region covers a contiguous row-key range; the system meta table tracks region locations.	One user-table region plus one meta region. No split or failover is demonstrated.
ZooKeeper	Coordinates HBase service state; it is not the task-data table.	Embedded in this standalone single-JVM setup.

Where a put goes

```
put -> RegionServer -> write-ahead log (WAL) + MemStore
                     -> flush -> HFile (StoreFile)
                     -> later compaction merges StoreFiles
get/scan -> RegionServer reads MemStore, BlockCache and HFiles
```

The WAL makes recent writes recoverable if the RegionServer fails before an in-memory flush. MemStore buffers cells; flush writes immutable HFiles; compaction consolidates files. BlockCache may serve frequently read file blocks. A get uses one exact key; a scan walks ordered key ranges. There is no general SQL WHERE clause on state:group in this station.

On the table's Regions view, MemSize reflects buffered cells, Storefile Size and Num.Storefiles reflect flushed files, and Compactions shows file-merging work. Tiny values may be rounded and no compaction may be active.

Standalone boundary

The container runs HMaster, RegionServer and ZooKeeper together and stores files under its local /var/lib/hbase volume. Production HBase commonly uses distributed storage and multiple servers, but this lab has no HDFS replication or node-failure survival. The Master and RegionServer are different roles even when they share this one JVM.

References: HBase data model and standalone mode, Master, RegionServer and Regions.

HBase: guided first run

Purpose and data model

This station stores the latest state of 40 synthetic affinity recompute tasks. HBase is a key-oriented, sparse column-family store: the row key chooses the primary access path, and each cell belongs to a family and qualifier. The lab writes one row per task, not the full lifecycle or an affinity score.

Piece	This station	What it teaches
Row key	EVENT#station-task-000000000	Keys sort lexicographically. The first five scan rows start with ASSET; a get uses one exact key.
Column family	state	The family is declared when the table is created and groups related cells.
Qualifiers / cells	state:status = done; state:group = demo-base-a	A cell has a value and write timestamp. The table retains one version, so a later put replaces the visible value.
Region	One affinity_tasks key range on one RegionServer	A region is a contiguous row-key range served by a RegionServer; this lab does not distribute it.

1. Open the Master UI

After ./stations.sh hbase up and ./stations.sh hbase run, open http://127.0.0.1:16011/master.jsp. No login is configured for this local page. Under Region Servers, expect one Normal server and two regions in total: one for the user table and one for hbase:meta. Under Tables > User Tables, open affinity_tasks in the default namespace; it is ENABLED with one region.

2. Read the table page

Table Schema shows the state family and VERSIONS = 1. Table Regions shows its OPEN region, server and request counters. The run makes 80 puts (two cells for each of 40 rows), so WriteRequests is about 80 after a fresh run; reads can change the ReadRequests count. The Master UI describes structure and health; it does not list the 40 row values. Its RegionServer link uses the container-only hbase:16030 address, which may not open from the host browser.

3. Inspect the actual rows

```
./stations.sh hbase run  # prints count 40 and five scan rows
printf "get 'affinity_tasks', 'EVENT#station-task-000000000'\n" | \
  docker compose -f stacks/hbase/compose.yaml exec -T hbase hbase shell -n
```

The get result has state:status = done and state:group = demo-base-a. The timestamp is the HBase write time, not the original task event time. Rerunning the fixture updates the same rows; it does not build an event history.

MyScoutee fit and limit

A possible large-scale read projection is latest recompute status by a known task key. It would need a backend-owned export and a row key designed for actual access patterns, perhaps including baseGroupId for group-scoped scans. The backend must still enforce authorization. This demo key is organized by target type, so it is not a drop-in key for the admin view. Mongo tasks and the backend worker remain authoritative.

Reference: Apache HBase data model and Getting Started.

Cassandra: concepts and write path

A query-shaped wide-column table

Cassandra distributes rows by a hash of their partition key. CQL resembles SQL, but a table is designed around known reads. The lab keyspace myscoutee_lab is a namespace with SimpleStrategy replication factor 1. Its table stores one final state per synthetic affinity task; it is not an event history or a score engine.

```
PRIMARY KEY ((target_type, day), task_id)
partition key:  (target_type, day) -> token / replica
clustering key: task_id -> sorted within that partition
```

Term	Meaning	In this lab
Keyspace / table	Namespace and replication policy / rows with typed columns.	myscoutee_lab / affinity_task_status
Partition / token	Rows sharing a partition key map to a token and its replicas.	USER + 2026-01-31 is one partition.
Clustering key	Orders rows within one partition and makes them unique there.	task_id sorts the task rows.
Regular columns	Stored values, not free general-purpose filter keys.	status, group_id and runtime_ms.
Node / DC / rack	Placement and failure-domain terms for a cluster.	One node; SimpleSnitch reports datacenter1/rack1.

A write and a read



A memtable flush creates immutable SSTables; background compaction merges them.

The coordinator routes a write to the replica chosen by the partition token. On that replica, the mutation enters the commit log and memtable; a later flush creates an immutable SSTable, and compaction merges SSTables. A read with the complete partition key can target the relevant partition. This one-node lab cannot show cross-node routing, replication or consistency tradeoffs.

Production boundary

RF 1 means one copy; UN in nodetool means Up/Normal, not fault tolerance. SimpleStrategy and SimpleSnitch are classroom defaults. A real deployment needs a multi-node topology, suitable replication and consistency choices, capacity tests, repair and operations planning. A target/day partition can grow too wide; time bucketing or sharding must follow measured read and write patterns.

References: Cassandra storage engine, CQL keys and production recommendations.

Cassandra: guided first run

1. Start, populate and inspect

```
./stations.sh cassandra up
./stations.sh cassandra run
docker compose -f stacks/cassandra/compose.yaml exec cassandra cqlsh
```

There is no web page or login in this station. The cqlsh prompt is its data interface. The run script creates the keyspace and table, writes 40 latest states, then prints five USER/day rows. A second run updates those same task IDs; it does not append the three lifecycle transitions.

2. Read schema and data in cqlsh

```
DESCRIBE KEYSPACE myscoutee_lab;
DESCRIBE TABLE myscoutee_lab.affinity_task_status;
SELECT task_id, status, group_id, runtime_ms
FROM myscoutee_lab.affinity_task_status
WHERE target_type='USER' AND day='2026-01-31' LIMIT 5;
```

The query uses both parts of the partition key. Expect five done rows, including task IDs station-task-000000001, -000000002, -000000003, -000000006 and -000000007. The exact runtime_ms values are synthetic. DESCRIBE shows task_id as the clustering key and group_id as an ordinary column.

3. Read node health from the host

```
docker compose -f stacks/cassandra/compose.yaml exec cassandra nodetool status
```

Expect one UN node in datacenter1. The image's SimpleSnitch reports rack1. The local keyspace uses SimpleStrategy with replication_factor 1, so the single node owns all tokens. SELECT count(*) over the entire table may return 40, but Cassandra warns about an aggregation without a partition key; use it only as a fixture check, never as a serving-query design.

MyScoutee fit and limit

A future read projection could answer high-volume latest task-status queries by a known target/day and task ID. The actual Admin Monitoring and Jobs views are group scoped: the demo's group_id is not in the partition key, so it cannot efficiently serve that access pattern. A real projection might key by (base_group_id, target_type, day) after measuring partition size, with backend authorization enforced before any read. Mongo tasks and the backend worker remain authoritative; Cassandra would hold a derived copy, not replace task execution.

Compare HBase's ordered row-key ranges with Cassandra's hashed partition placement and clustering order inside a partition. Both station tables hold the same 40 final states, but they favor different queries. Finish with ./stations.sh cassandra down; the volume remains for the next visit.

References: CQL query and filtering rules and Cassandra quick start.

Cassandra: cluster model and MyScoutee fit

Peer nodes, replicas and consistency

Any Cassandra node can coordinate a client operation; there is no dedicated data master. The partition key maps to a token and the keyspace policy chooses replicas. Gossip shares cluster membership. This one-node RF 1 station demonstrates neither replication nor failover.

Concept	Three-node RF 3 example	Current station
Replicas	Each partition has three copies on distinct nodes.	One copy on the only node.
Consistency ONE	The coordinator waits for one replica response; a write is still sent to all replicas.	One response from the only node.
Consistency QUORUM	A majority of RF 3 is two responses. Read and write quorums overlap.	No multi-replica behavior to observe.
Topology	Use NetworkTopologyStrategy and planned failure domains for production.	SimpleStrategy + SimpleSnitch for learning.

What each store is optimized to teach here

Store	Useful access pattern	MyScoutee boundary
Cassandra	Known partition-key reads and sustained writes across a replicated cluster.	Possible derived status projection only after measured demand.
MongoDB	Flexible application documents and product-owned task lifecycle.	Existing authoritative affinity task store and worker inputs.
HBase	Ordered row-key access and scans over key ranges.	Separate row-key experiment; standalone lab has no HDFS.
Hive	Batch SQL over historical tabular data and files.	Activity cohort analysis, not per-request task serving.

A possible integration, not a running pipeline

```
backend-owned task transition -> approved Kafka topic
  -> new stream projection -> Cassandra status-by-group/day table
  -> authorized backend API -> Admin Jobs / Monitoring
```

Current Flink computes ten-second rollups for ClickHouse; it neither writes Cassandra nor preserves latest task state. A real projection needs a separate idempotent consumer, stable IDs and ordering for late transitions, group-aware keys, retention and deletion rules. The backend retains task execution and authorization. This station imports 40 synthetic final states directly through CQL; no Kafka, Flink or product connection exists.

When to consider it

Measure lookup volume, write rate and MongoDB latency first. Only a demonstrated bottleneck justifies testing a replicated, group/day projection and worst-case partition size. Cassandra does not replace MongoDB's authoritative writes, Hive's analytical SQL or Kafka's event log; CQL lacks relational joins and general cross-partition transactions.

References: Cassandra replication and consistency, architecture overview and data modeling.

Airflow: guided first run

1. Open and sign in

Run ./stations.sh airflow up from the repository root, then open http://127.0.0.1:8096. Sign in as admin. The up command copies the current generated password into the repository's ignored .local/state/airflow-passwords.json file (mode 600).

```
cat .local/state/airflow-passwords.json
```

The JSON key is the username and its value is the password. Up refreshes this local copy; down retains it and the Airflow Docker volume, while reset removes both. Keep the file out of Git and shared documents.

2. Find the DAG

The Home page shows service health. Open Dags and search for myscoutee_activity_cohort. Examples are disabled, so this is the one lab DAG. Its schedule is None: it does not start on a timer. It may appear paused, and earlier test runs may already be listed because the Airflow volume survives down/up.

3. Read the pipeline before running it

Open the DAG and select Graph to see extract -> validate -> publish. Code shows the Python definition. Extract counts synthetic activity events and joins by city/mode; validate checks the registration/onboarding/join ordering and join sum; publish writes activity_cohort.json. Grid and Runs show previous executions, while a task's Logs and XCom tabs show its execution details and returned value.

4. Trigger and watch a new run

Resume the DAG if it is paused, then use Trigger to start a manual run. In Runs or Grid, watch the run progress to success; open Graph for that run to see all three tasks succeed. There is no automatic schedule. Alternatively, ./stations.sh airflow run executes a deterministic local DAG test for logical date 2026-01-31 and prints the report in the terminal.

5. Check the published report

```
docker compose -f stacks/airflow/compose.yaml exec -T airflow \
  cat /opt/airflow/output/activity_cohort.json
```

With the default seed, expect 120 registrations, 103 onboarding events and 71 joins; the join breakdown is Bratislava_Casual 55 and Praha_Casual 16. These are synthetic counts. The JSON path is inside the persistent Airflow volume, not on the host.

6. Stop this station

```
./stations.sh airflow down
```

Down stops the container but retains its Airflow volume and run history. The DAG is a batch orchestration example, not a live MyScoutee backend integration.

Airflow, GitLab and admin views

Four systems show flows or jobs, but each is optimized for a different question:

System	Optimized for
MyScoutee Admin Monitoring	Observing product operation: aggregate node/edge counts and health on four-node flow maps. The arrows summarize backend signals.
MyScoutee Admin Jobs / Processes	Controlling product-owned scheduled work: timing, manual start, suspension, progress and run history.
GitLab CI pipelines	Validating and delivering source changes: dependent build, test, package, deploy and release jobs, with manual gates, run states and logs.
Airflow DAGs	Orchestrating data workflows: scheduled or manual multi-step runs, data-readiness checks, retries, task logs and historical backfills.

GitLab CI is the closest operational analogy to Airflow: both execute a dependency graph of jobs and expose run status and logs. GitLab is organized around code delivery; Airflow is organized around data processing. The admin Monitoring map resembles Airflow's Graph visually, while admin Jobs resembles its scheduling and run controls. The proposed analytics DAG would run separately from existing product jobs and affinity workers.

DAG, dependencies and task data

Concept	Read it on the Airflow screen
DAG	Directed acyclic graph: tasks and one-way dependencies with no cycle. Graph shows which task can start after which predecessor succeeds.
Task and run	A task is one unit of work; a DAG run is one execution for a logical date or manual trigger. Grid and Runs show history; Logs explain a selected task attempt.
XCom	Small cross-task values such as a count or an output path. The XCom tab shows values returned by this lab's Python tasks; it is not a channel for full datasets.
Dataset handoff	A real Spark output would be stored in shared object storage, with its location passed as small metadata. A dependency arrow only controls order; it does not itself move rows.

In this single-container lab, the tasks read local fixture files and exchange small values. A distributed deployment needs explicit shared storage for large outputs. The Admin Monitoring arrows summarize product metrics; they are not Airflow task dependencies or data transfers.

Reference: Airflow DAGs and XComs.

Airflow: MyScoutee data use cases

Use case A: daily registration-to-participation cohorts

Question: among people who registered on a given day, how many completed onboarding and joined their first event within 7 or 30 days? Break down approved aggregate results by city and event mode. This extends the current admin counters with a defined historical cohort window.

```
approved backend export -> validate partition and schema
  -> Spark cohort calculation -> Parquet snapshot
  -> ClickHouse aggregate -> compare with admin statistics
```

Airflow task	Concrete responsibility and failure signal
Export ready	Wait for a complete, backend-owned daily export with stable event IDs and an approved pseudonymous cohort key. Fail if the partition is missing.
Validate	Check schema version, duplicate IDs, date coverage and required fields; compare source totals with AdminStatsService where definitions align. Fail before publishing incomplete data.
Compute	After each 7- or 30-day cohort window closes, launch Spark to link registration, onboarding and first join; write versioned Parquet partitions to the S3-compatible lake.
Publish and reconcile	Load only aggregate counts into ClickHouse, compare them with product definitions and record the run result. Rerunning a day must replace or deduplicate its prior output.

A failed cohort partition stays visible in Runs, Grid and task Logs. A corrected source partition can be replayed after the operator checks the cause. Late events need an agreed revision window. Airflow would own this batch schedule and dependency chain; the backend remains the owner of product events and admin metrics.

Use case B: affinity recompute history

Export only approved task transitions and timing fields from the backend owner. A daily Airflow job can validate the export, calculate pending/running/done/failed counts and queue wait/runtime distributions by target type, then publish historical aggregates. Compare them with AffinityRecomputeMetricsService and the Admin Monitoring Matching view. The existing Mongo task queue and AffinityRecomputeWorkerService continue to execute real recomputations.

Current lab boundary

The runnable myscoutee_activity_cohort DAG only reads a local synthetic JSONL fixture, checks its counts and writes one JSON report. Its schedule is None. It does not call the backend, Spark, the S3 lake or ClickHouse. A real integration first needs a reviewed export contract, minimal data, deletion/retention rules, source-to-report reconciliation and replay tests.

NiFi: guided first run

Purpose and MyScoutee fit

NiFi designs and monitors a visual dataflow for ingesting, inspecting and routing individual files or messages. For MyScoutee, a proposed use is intake of a backend-owned, approved event export: check its agreed schema and required fields, route accepted records to a downstream data pipeline, and isolate invalid records for review. This is a possible integration, not an active backend path.

Three building blocks on this canvas

Building block	What it means here
FlowFile	One moving data item: original JSON content plus attributes. Each input file becomes a FlowFile; EvaluateJsonPath adds event_key as an attribute.
Processor	An active step that reads, extracts, routes or writes. GetFile, EvaluateJsonPath, RouteOnAttribute and the two PutFile instances are the five processors.
Connection	A directed link and queue between processors. Its label selects the route; counters show queued items and bytes. If downstream stalls, backpressure can pause the source.

1. Open the canvas

From the repository root, run ./stations.sh nifi up and ./stations.sh nifi run. Open https://localhost:8444/nifi and sign in as labuser / lab-only-password-123. The local certificate is self-signed, so the browser may ask for a one-time exception. The installer creates and starts five processors on the root canvas.

2. Follow the two routes

```
GetFile -> EvaluateJsonPath -> RouteOnAttribute -> Accepted PutFile
                             |-> Rejected PutFile
```

GetFile consumes JSON from /opt/lab/inbox. EvaluateJsonPath copies $.event_key into an attribute while preserving the JSON. RouteOnAttribute sends a non-empty key to valid and a missing or empty key to rejected. The two PutFile processors write to separate host-mounted directories.

3. Read the flow state and provenance

Inspect each processor's running state and the counters on its connections. Queues can already be empty because both files finish quickly; this is expected. In the Global Menu, open Data Provenance and inspect events for valid.json and invalid.json. Open an event's details or lineage to follow where each FlowFile went. A FlowFile has both content and attributes; routing here uses the extracted attribute, while PutFile retains the original content.

4. Verify outputs and repeat

```
ls build/stations/nifi-output/valid/
ls build/stations/nifi-output/rejected/
python3 producer/export.py   # regenerate consumed input files
```

This lab checks only whether event_key is non-empty in two synthetic files: expect valid/valid.json and rejected/invalid.json. It does not validate a full event envelope or publish to Kafka. GetFile picks up regenerated files. Finish with ./stations.sh nifi down; after a fresh container, rerun ./stations.sh nifi run.

11. Observe the core lab

Activity stream: Kafka to ClickHouse

```
./lab.sh up
./lab.sh seed --users 5000 --days 30 --seed 42
./lab.sh topics
./lab.sh groups
./lab.sh query "SELECT event_key, count() FROM activity_events
  GROUP BY event_key ORDER BY count() DESC"
```

Open http://127.0.0.1:8094 and watch the activity cards and city/mode breakdown update after seeding. The ClickHouse query is the exact-count check. Kafka has no dedicated browser UI here: topics and consumer groups are inspected by the two CLI commands. In a replay exercise, compare counts and consumer offsets before and after adding a second seed.

Affinity stream: Flink

```
./lab.sh stream-up
./lab.sh affinity-seed --tasks 1000 --seed 42
./lab.sh query "SELECT target_type, sum(pending), sum(done),
  sum(failed) FROM affinity_rollups FINAL GROUP BY target_type"
```

Open http://127.0.0.1:8081. Find myscoutee-affinity-recompute-rollups; inspect its task managers, running vertices and checkpoints. After a 10-second window closes, the dashboard's affinity panel and the ClickHouse query should show task-state totals. Flink measures synthetic queue telemetry; it does not calculate affinity scores.

Historical batch: Spark to the S3 lake

```
./lab.sh batch
./lab.sh history-up
AWS_ACCESS_KEY_ID=lab AWS_SECRET_ACCESS_KEY=lab-only-secret \
  aws --endpoint-url http://127.0.0.1:8333 s3 ls \
  s3://myscoutee-lake/daily_activity/ --recursive
./lab.sh down
```

The batch prints its aggregate row count; a second run overwrites daily_activity. Open http://127.0.0.1:18080 for Spark's completed job, stage and task metrics, and http://127.0.0.1:8888/buckets/myscoutee-lake/daily_activity/ for the SeaweedFS object browser. The S3 listing is a separate authenticated API check. Stop the optional History Server with ./lab.sh history-down to free memory.

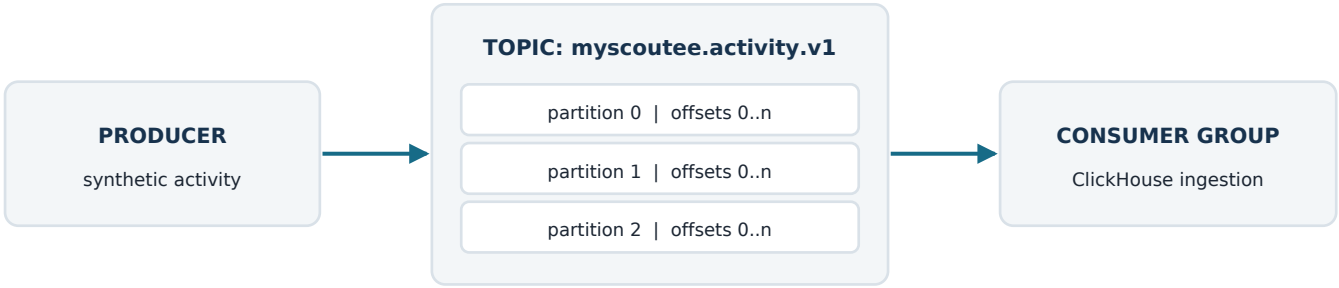
When an expected result is absent

Check ./lab.sh status or ./stations.sh STATION status, then inspect the relevant container logs with docker compose. Kafka-to-ClickHouse ingestion is asynchronous, so allow a few seconds after seeding. Flink's dashboard numbers appear after the first 10-second window. Stop the active project with down before starting another station; down preserves its volumes.

Kafka: concepts and MyScoutee role

A retained event log

Kafka stores records in named topics. A producer appends records; consumers read them independently. Reading does not remove a record. This lab uses one KRaft broker, with broker and controller roles in the same container, so it teaches the API and offsets rather than broker failover.



The actor_id key selects a partition; ordering and offsets are local to that partition.

Term	Meaning	This lab
Topic	Named append-only record stream.	activity.v1, affinity.tasks.v1 and affinity.rollups.v1.
Partition	Ordered slice of a topic; offsets are separate per slice.	Three per topic, all on broker 1.
Record / key	Value plus optional key; the key selects a partition.	JSON activity value, actor_id as key.
Offset	A record's position within one partition.	Log-end is the next append position.
Consumer group	Readers sharing one group split assigned partitions.	myscoutee-clickhouse-v1 ingests activity.
Lag	Log-end offset minus committed current offset.	Zero after ClickHouse catches up.
Retention	How long records remain available for later reads.	Seven-day broker policy in this Compose file.

Ordering, replay and MyScoutee fit

The producer keys activity records by actor_id, so records for one synthetic actor go to the same partition and retain their order there; Kafka does not impose one total order across all three partitions. A new consumer can read retained records from an earlier offset without stopping ClickHouse. A future backend-owned, post-commit export could fan out approved profile and event signals to independent analytics consumers. The current Redis admin telemetry path and Mongo product state remain backend-owned; this lab has no live connection.

One broker and replication factor 1 mean there is no redundant copy. Kafka transports and retains events; it does not calculate affinity scores or provide the historical SQL view. The ClickHouse dashboard is downstream of Kafka, not a Kafka management UI.

References: Apache Kafka introduction and Kafka 4.1 Docker guide.

Kafka: guided first run

1. Start the core and inspect its topics

```
./lab.sh up
./lab.sh topics
docker compose -f stacks/core/compose.yaml exec -T kafka \
  /opt/kafka/bin/kafka-topics.sh --bootstrap-server kafka:9092 \
  --describe --topic myscoutee.activity.v1
```

There is no Kafka web UI, host port or login in this lab. The broker is reached inside Docker. The topic description shows three partitions, replication factor 1 and broker 1 as each leader and in-sync replica. The core start also runs ClickHouse, SeaweedFS and the dashboard; Spark and Flink are still off.

2. Publish a small synthetic batch

```
./lab.sh query 'SELECT count() FROM activity_events'
./lab.sh seed --users 120 --days 30 --seed 42
./lab.sh query 'SELECT count() FROM activity_events'
```

The seed prints 306 synthetic activity records. With retained volumes, the table count rises by 306 after ClickHouse catches up; it need not start at zero. On the documented visit it rose from 700 to 1006. Each seed has a new run_id and event IDs, so repeating it appends another batch. The dashboard at http://127.0.0.1:8094 shows ClickHouse's downstream counts, not the Kafka log.

3. See a record and its position

```
docker compose -f stacks/core/compose.yaml exec -T kafka \
  /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server kafka:9092 --topic myscoutee.activity.v1 \
  --from-beginning --max-messages 1 --property print.key=true \
  --property print.partition=true --property print.offset=true
```

The output shows a partition number, its offset, an actor_id key and one JSON event value. It may be from an older retained batch; --from-beginning reads the earliest available record. The three log-end offsets increased from 223/239/238 to 336/321/349 in this visit: their combined increase is 306. Your offsets may differ.

4. Check the ClickHouse consumer group

```
docker compose -f stacks/core/compose.yaml exec -T kafka \
  /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server kafka:9092 --describe \
  --group myscoutee-clickhouse-v1
```

Each partition has CURRENT-OFFSET, LOG-END-OFFSET and LAG. Lag reached 0 on all three after this seed. A nonzero value means the group has not committed up to the log end yet; it is not an event count from MyScoutee production. Keep the core running for the next ClickHouse visit; ./lab.sh down preserves its volumes.

Reference: Kafka quick start and topics and consumers.

Kafka: consumer lag in action

The first run shows a caught-up consumer. To see Kafka's retained log, pause only the downstream ClickHouse consumer, publish a few more synthetic events, and compare each partition's log end with the group's committed position. Kafka keeps the records while the consumer is absent. This is an operational replay lesson, not a product data feed.

1. Pause the downstream services

```
docker compose -f stacks/core/compose.yaml \
  stop dashboard clickhouse lake
./lab.sh status  # only Kafka should be running
```

The topic-creation helper has already exited successfully. Stopping ClickHouse does not delete its volume, the Kafka topic or the committed consumer-group offsets. The dashboard and object store are also stopped to save laptop memory.

2. Publish three records and inspect lag

```
./lab.sh seed --users 1 --days 1 --seed 42
docker compose -f stacks/core/compose.yaml exec -T kafka \
  /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server kafka:9092 --describe \
  --group myscoutee-clickhouse-v1
```

This seed produced event.created, profile.registered and profile.onboarding. On the documented visit, the ClickHouse group had no active members and showed:

Partition	Before log end	After log end	Committed current	Lag
0	336	336	336	0
1	321	323	321	2
2	349	350	349	1

The three new offsets are split 0 + 2 + 1 across partitions; they are not a single global counter. Your absolute offsets depend on retained runs. LAG = LOG-END-OFFSET - CURRENT-OFFSET for each partition, so the group's total pending count here is 3.

3. Read without moving the ClickHouse group

```
docker compose -f stacks/core/compose.yaml exec -T kafka \
  /opt/kafka/bin/kafka-console-consumer.sh \
  --bootstrap-server kafka:9092 --topic myscoutee.activity.v1 \
  --partition 1 --offset 321 --max-messages 2 \
  --property print.key=true --property print.offset=true
```

In this run the two records at offsets 321 and 322 share one synthetic user key and are registered then onboarding. This separate console read does not advance myscoutee-clickhouse-v1. With a different retained volume, choose a partition and starting offset from your own log-end comparison.

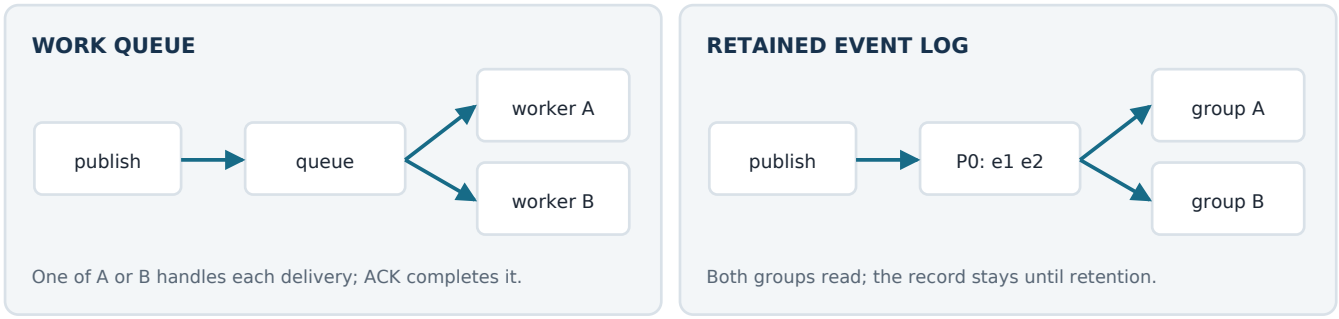
Keep Kafka running and the ClickHouse group behind for now. Starting ClickHouse on the next step should let it resume from its saved offsets; then check that the lag returns to 0 and the raw table gains three rows. No Kafka web UI or login exists in this lab.

Reference: Apache Kafka topics, partitions and consumers.

Kafka, brokers and work queues

Kafka is a message broker

A broker is a service between producers and consumers. Kafka brokers accept, store and serve records; Kafka is an event-streaming platform built from brokers, not an alternative to the word 'broker'. The useful architectural comparison is a conventional work queue versus a retained, partitioned event log. RabbitMQ has both queue and stream types, so the table compares models, not entire products.



Decision	Work-queue model	Kafka topic + ordinary consumer group
Delivery	Workers compete for queued jobs; acknowledgement completes a delivery and a failed unacknowledged job can be redelivered.	Consumers fetch partition records by offset; committing an offset records group progress. Reading leaves the record in the topic.
History	A completed queue message is normally removed from the queue's available work.	Retention controls how long the record remains, even after consumption. Another group can replay retained history.
Fan-out	Use separate bound queues or another routing setup for independent work.	Independent groups each have their own progress on the same topic.
Parallelism	Workers compete for individual queued messages.	Within one ordinary group, each partition has one active owner; three partitions permit at most three active readers for that topic.
Failure unit	Acknowledge, retry or reject an individual delivery according to queue policy.	An offset is a position; retrying one bad record may stall its partition unless the consumer handles or isolates it.

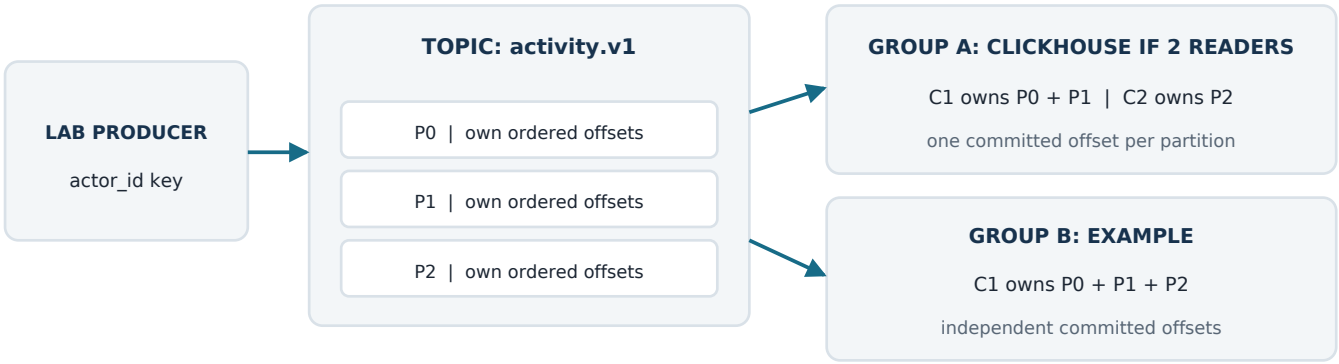
Concrete MyScoutee example

An affinity recompute task is a command to perform work and update authoritative MongoDB state. Its current durable task service and worker own that lifecycle. By contrast, a post-commit 'task finished' fact can be exported for several projections: a metrics consumer, a ClickHouse analytics consumer and perhaps an audit consumer. Each must use a distinct group ID. Kafka would distribute facts to them; it would not make the affinity score or become the task's source of truth.

A stream is not an infinite archive. A consumer cannot replay records that expired under retention. RabbitMQ streams also support retention and replay; Kafka 4.2 adds share groups for queue-like consumption, while this lab uses Kafka 4.1.2 ordinary consumer groups.

References: Kafka introduction; RabbitMQ queue and stream comparison.

Kafka architecture and consumer groups



What happens to one event

The producer sends a key and value to a topic. The key chooses a partition; this lab keys by actor_id, so one actor's activity stays ordered within one partition. A broker appends the record at that partition's next offset. The ClickHouse group reads it, writes a projection and commits its next offset. A separate cohort group can still read the same record. Offsets have meaning only within a partition; there is no topic-wide sequence.

Component	Job and consequence
Broker / controller	A broker serves records. In KRaft, controllers manage cluster metadata. This laptop combines both roles in one process.
Leader / replicas	A production partition can have a leader and follower replicas; in-sync replicas protect availability according to replication and acknowledgement settings. This lab has one broker and replication factor 1.
Group coordinator	Tracks group membership and partition assignment. A member joining, leaving or timing out can trigger a rebalance.
Offset commit	A group's durable next-read position. Commit after a successful side effect to avoid skipping work; a crash before commit can cause redelivery.
Lag	For a partition, log-end offset minus group's committed offset. It measures unread positions, not processing latency or correctness; check age and errors too.

Rebalance example

For the lab's three partitions, one ClickHouse-group member can own P0, P1 and P2. Adding a second member redistributes them, for example P0+P1 to C1 and P2 to C2. A fourth member would sit idle for this topic. Rebalance temporarily changes ownership; a correct consumer finishes or safely retries in-flight work before committing. A separate group gets its own copy of the partition assignments and independent offsets.

Failure and replay

If ClickHouse stops, its committed offsets stay put while log ends advance: lag rises, as on page 31. When it restarts before retention expires, it resumes. If it writes a row and crashes before committing, it may write that record again. Use stable event IDs and an idempotent sink or deduplication. Resetting a group to earlier offsets is an explicit backfill operation; first decide whether the target table should be rebuilt or deduplicated.

References: Kafka introduction; Kafka 4.1 consumer configuration.

A MyScoutee Kafka design

Two bounded candidate streams

Candidate	Backend-owned event	Independent read models
Participation analytics	After committed profile registration/onboarding or event membership change: event_id, actor_id, event_type, occurred_at, schema_version; export only approved fields.	ClickHouse live counts by day/city and Spark cohort reconstruction. Key by actor_id when per-user order matters; define how membership changes correlate with an event.
Affinity operations	After a durable recompute task changes state: task_id, target_type, state, occurred_at, schema_version; never treat this signal as the score input.	Flink windows for queue wait, completion and failure trends; ClickHouse operational history. Key by task_id for task transition order.

Method: introduce a read model without losing a write

Step	Required design decision
1. Define the fact	Name the source owner, event time, stable ID, key, versioned payload and privacy classification. Distinguish a command ('recompute') from a fact ('recompute finished').
2. Publish after commit	Use a transactional outbox or an equivalent durable post-commit mechanism; otherwise a database commit followed by producer failure loses the event. Measure duplicate delivery and export delay.
3. Choose topology	Set partitions from measured throughput and key skew; separate consumer groups per independent projection, replicas and retention from recovery needs. Three partitions on one lab broker are for learning, not a capacity prescription.
4. Process safely	Assume at-least-once side effects unless the full source-to-sink path is proven otherwise. Validate schema versions, deduplicate by event_id, quarantine poison records and alert on lag age, errors and missing source events.
5. Prove replay and deletion	Rebuild a disposable projection from retained data; compare counts with backend truth. Define correction, backfill, retention and personal-data deletion rules before product export.

Adoption boundary

The current MyScoutee backend owns Mongo task state, affinity calculation and Redis admin telemetry. The lab's synthetic topics demonstrate a possible downstream integration only. Kafka becomes worthwhile when independent consumers, replay or measured event volume justify the operational cost. It does not replace the current task worker or the admin monitoring flow by itself. No production exporter, outbox, authentication or deletion pipeline is implemented in this repository.

References: Kafka introduction; Kafka 4.1 documentation.

ClickHouse: from events to answers

An analytical read model

ClickHouse is the lab's column-oriented analytical database. Its role is to answer grouped questions across many event rows. The MyScoutee backend still owns product writes and the meaning of each metric. A row entering Kafka is not automatically a dashboard number: it must be consumed, stored, and queried with a chosen definition.



Kafka retains the log; the MergeTree table stores the queryable copy on the ClickHouse volume.

Piece	What it does here	What to inspect
Kafka Engine table	activity_queue reads JSONEachRow records from myscoutee.activity.v1 as group myscoutee-clickhouse-v1. It is an ingestion adapter, not the durable analytical table.	system.tables engine = Kafka; group lag in Kafka.
Materialized view	activity_ingest consumes batches from activity_queue and inserts them into activity_events. It is an insert-time data path, not a dashboard visualization.	system.tables engine = MaterializedView.
MergeTree table	activity_events stores nine event fields. PARTITION BY month groups storage; ORDER BY occurred_at, event_key, event_id sorts parts for range-oriented reads.	SELECT count(), event_key, day, city; inspect CREATE TABLE.
Dashboard	Read-only HTTP service runs SQL against activity_events and affinity_rollups, then redraws cards every ten seconds.	http://127.0.0.1:8094 and /api/metrics.

Storage and query terms

Columnar storage reads only the columns needed by an aggregate. MergeTree writes sorted parts and merges them in the background; its sparse index helps skip ranges aligned with the sort key. Monthly partitioning is a storage and pruning choice, not a Kafka partition or a MyScoutee tenant. LowCardinality(String) is used for repeated labels such as city and mode. GROUP BY forms buckets; count() counts rows, while uniqExact(event_id) counts distinct event IDs. Neither proves the source emitted every fact.

The raw MergeTree table does not enforce unique event_id. A replay or retry can duplicate rows, so the dashboard's distinct-ID formulas and a production deduplication policy matter. An insert-time materialized view also does not automatically reconstruct all past rows if it is created after the source has already been consumed.

References: ClickHouse Kafka table engine; MergeTree reference.

ClickHouse: guided first visit

1. Resume the waiting Kafka records

```
docker compose -f stacks/core/compose.yaml up -d clickhouse dashboard
./lab.sh query 'SELECT count(), uniqExact(event_id) FROM activity_events'
docker compose -f stacks/core/compose.yaml exec -T kafka \
  /opt/kafka/bin/kafka-consumer-groups.sh \
  --bootstrap-server kafka:9092 --describe \
  --group myscoutee-clickhouse-v1
```

On the documented visit the table grew from 1006 to 1009 rows after the three pending Kafka records were consumed; all three partition lags returned to zero. With retained volumes or another seed, your absolute counts differ. A zero lag only says the consumer group caught up; check the table and metric definitions separately.

2. See the three ClickHouse objects

```
./lab.sh query "SELECT name, engine FROM system.tables
  WHERE database='myscoutee_lab' ORDER BY name"
./lab.sh query "SELECT event_key, count() AS rows FROM
  activity_events GROUP BY event_key ORDER BY rows DESC"
```

Find activity_queue (Kafka), activity_ingest (MaterializedView), and activity_events (MergeTree). The event query groups stored rows by event type; it does not query the Kafka log. On this visit, the stored table had 421 profile.registered, 343 profile.onboarding, 207 event.member.joined, 23 event.member.removed and 15 event.created rows.

3. Read the dashboard at http://127.0.0.1:8094

Panel	Meaning and limit
Top cards / profile to participation	Distinct actors with registration, onboarding or join events, plus distinct created activities. These are independent all-history counts, not a verified same-person conversion cohort.
Cities / event modes	Registration and join counts by city; created and joined event counts by mode. Check whether city and mode describe the same business dimension before using the numbers.
Last 30 days	Daily event counts by occurred_at in the trailing 30-day window. Older records remain in the raw table but do not appear in this chart.
Affinity panel	Reads retained synthetic Flink rollups from an earlier run if present. Flink is off in this visit; this panel is not live backend worker health.

MyScoutee decision boundary

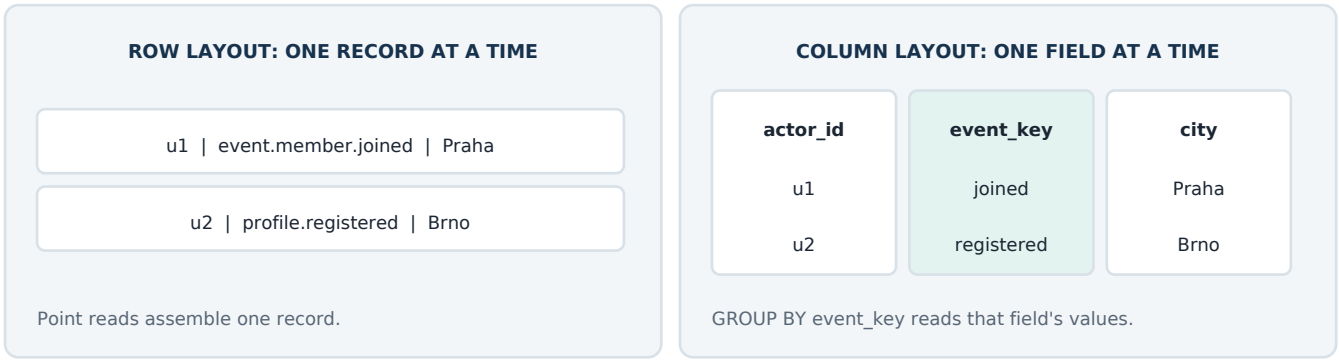
A reviewed backend export could feed activity and affinity-operation analytics when repeated aggregates outgrow current reporting. Keep the backend's product state and admin metrics authoritative until event contracts, consent, deletion, duplicate handling, metric definitions and reconciliation are proven. This local dashboard is a learning surface over synthetic data; it does not replace MyScoutee admin monitoring.

Access: dashboard 127.0.0.1:8094; ClickHouse HTTP 127.0.0.1:8126; CLI user lab, password lab-only. The HTTP endpoint is an API, not a ClickHouse browser console. Kafka and ClickHouse remain running for this visit; SeaweedFS, Spark and Flink are stopped.

ClickHouse: why columnar analytics

OLAP and OLTP are workload shapes

OLTP (online transaction processing) handles current product state: small reads and writes, such as changing one profile or completing one durable affinity task. OLAP (online analytical processing) scans many historical rows to answer grouped questions, such as daily joined profiles by city. MyScoutee's backend remains the operational source; this lab's ClickHouse is an OLAP read model.



A row-oriented store keeps fields of one record together, which suits point reads and updates. A column-oriented store keeps values of the same field together, so an aggregate can read a few columns over many records and compress repeated values efficiently. The picture is a layout model, not a literal view of MergeTree's files. ClickHouse still has to read every column used by SELECT, WHERE or GROUP BY.

Try an analytical query on the actual lab schema

```
./lab.sh query "SELECT toDate(occurred_at) AS day,
  uniqExactIf(actor_id, event_key = 'event.member.joined')
  AS joined_profiles FROM activity_events
  WHERE occurred_at >= now() - INTERVAL 30 DAY
  GROUP BY day ORDER BY day DESC LIMIT 5"
```

This reads occurred_at, actor_id and event_key, groups rows by day, and counts distinct joining actors within each day. One actor may appear on multiple days; adding daily figures does not give a 30-day unique total. The dashboard's top cards use all retained ClickHouse rows, while its trend chart filters to the last 30 days.

What the MergeTree layout changes

Each insert creates a sorted data part; background merges combine parts. This table uses PARTITION BY toYYYYMM(occurred_at), so a matching time filter can exclude whole months. Within a part, ORDER BY (occurred_at, event_key, event_id) and a sparse index help skip irrelevant time ranges. Neither the monthly partition nor the sorting key guarantees event_id uniqueness. Large scans and aggregations are the reason to test ClickHouse; this 1009-row lab is too small to prove a speed advantage.

References: ClickHouse OLTP/OLAP explanation; MergeTree reference.

ClickHouse: choosing its role

Compare the question before the product

System	Best question in this MyScoutee context	What the lab shows
MongoDB backend	What is this profile's or affinity task's current authoritative state?	Existing backend owner; this lab does not write product state.
PostgreSQL (contrast)	How can a relational product record be updated in one transaction?	General OLTP comparison from the shared explanation; it is not a lab station or the current product owner.
ClickHouse	How many approved events or distinct actors by day, city, event mode or task status?	Kafka-fed activity_events, SQL aggregates and dashboard.
Hive / file lake	What does a reproducible historical cohort or partitioned file scan show?	Isolated Hive station and separate Spark-to-Parquet exercise.
Cassandra / HBase	What is the value for a known partition or row key at scale?	Isolated key-oriented stations; their wide-column model is not ClickHouse's columnar scan layout.
Elasticsearch	Which documents or log lines match a search phrase or relevance query?	Separate MyScoutee Elastic repository; overlapping aggregates do not make search equivalent to ClickHouse SQL.

Candidate MyScoutee analytics

Question	Approved fact needed	Definition to agree before adoption
Participation trend	Registration, onboarding and event membership changes.	Unique actors versus transitions; time window, event time, removals and late arrivals.
Event demand	Event created and member joined facts with a permitted mode/city dimension.	Whether mode and city belong to the event or actor; how edits and deletions change past counts.
Affinity operations	Recompute task state transitions and timestamps, without profile payloads.	Queue wait, run time, failures and replays reconciled with backend task metrics.

The shared explanation also suggests player-view, scout, video and search events. Those are generic examples, not events emitted by this lab. Do not count or promise them as MyScoutee metrics without a verified product event contract. The lab currently emits profile and event participation facts plus synthetic affinity task states; it does not export live MyScoutee data.

Decision rule

Introduce an analytical copy when measured dashboard or historical aggregation needs justify the extra pipeline. Define a backend-owned, post-commit export; minimize fields; plan retention and deletion; deduplicate retries; and reconcile counts against the backend. ClickHouse can complement MongoDB and search, but a functioning synthetic dashboard alone is not evidence to replace either.

References: ClickHouse OLAP and wide-column distinction; shared ClickHouse explanation reviewed for this section.

Spark: a historical batch job

The data path in this repository

Spark reads the Kafka activity topic as a bounded batch from the earliest still-retained offsets to the latest offsets visible at job start. It parses JSON events, discards rows without event_id or event_key, deduplicates by event_id, groups by day/city/mode/event_key and writes a Parquet dataset to the S3-compatible SeaweedFS bucket. This is a new read of history on every run, independent of the ClickHouse consumer group's committed offsets.



One on-demand batch reads retained offsets, shuffles for distinct/grouped results, and overwrites the output prefix.

Spark term	Meaning in this local job
Driver	The process running spark_batch.py: it builds the plan, starts actions and reports the result.
Executor / task	Spark runs task work in local[*] mode within an ephemeral container; there is no long-lived YARN or Kubernetes cluster here.
DataFrame	Distributed, schema-aware rows. Kafka first supplies binary key/value metadata; from_json(value) extracts the nine event fields.
Transformation	filter, dropDuplicates, withColumn, groupBy and aggregations describe a plan. They execute when an action needs their result.
Shuffle	Redistribution needed for global event-ID deduplication and grouping. It can cost memory, network and disk at larger scale.
Action	write.parquet executes the output plan; readback count() checks that the written dataset can be read.

What is being counted

One output row represents a (day, city, mode, event_key) combination, not one MyScoutee user or one Kafka record. The events column counts deduplicated input rows in that group; actors counts distinct actor_id values within it. Summing actors across groups can count the same person more than once. The current script does not compute registration-to-first-join cohorts or affinity scores.

Spark is an execution engine. Parquet is an output file format; SeaweedFS is object storage; Hadoop S3A is the client connector used to reach it. The separate Hadoop station's HDFS, YARN and MapReduce services are not part of this batch run.

References: Spark 4.0.4 Kafka source; Spark 4.0.4 Parquet.

Spark: guided first batch

1. Leave only the required services running

```
docker compose -f stacks/core/compose.yaml stop dashboard clickhouse
docker compose -f stacks/core/compose.yaml up -d lake
./lab.sh status  # Kafka and lake should be running
```

The on-demand Spark container runs only during the next command and is removed afterward. This keeps memory use lower than running ClickHouse, the dashboard, Flink and Spark together. The Kafka topic and SeaweedFS volume retain their data.

2. Run the batch and identify its result

```
./lab.sh batch
```

The documented run ended with: 'Wrote 259 daily aggregate rows to s3a://myscoutee-lake/daily_activity'. Spark read that Parquet output back and counted its grouped rows. This number is not 259 source events; the source included 1009 activity rows on this visit. Your result varies with Kafka retention and later seeds. The first run may spend time resolving Spark/Kafka/Hadoop S3A packages.

3. Inspect objects without an AWS CLI

```
printf 's3.bucket.list\nfs.ls /buckets/myscoutee-lake/\n' | \
  docker compose -f stacks/core/compose.yaml exec -T lake \
  weed shell -filer localhost:8888
```

The first command lists the bucket; the second lists its top-level key prefix. The S3 API is at 127.0.0.1:8333. After the batch, run ./lab.sh history-up and open the Spark History Server at http://127.0.0.1:18080. It rebuilds the completed application's UI from its persistent event log; runs made before event logging was enabled do not appear. The active 4040 UI closes when the SparkContext ends, even if its container were kept alive.

Rerun and correctness

startingOffsets=earliest and endingOffsets=latest make every batch scan retained Kafka history again. dropDuplicates(event_id) removes duplicate IDs in that batch but does not resolve two conflicting payloads with the same ID; production must detect that case. write.mode('overwrite') replaces daily_activity; a second run does not preserve the earlier output as another snapshot. There is no durable checkpoint, publication marker, schema registry, or production reconciliation in this job. A safe production design would write to a new immutable run prefix, verify it, then publish a pointer to the accepted result.

MyScoutee fit

A reviewed backend export could let Spark rebuild historical participation summaries or person-linked cohorts after a definition changes. The current output is only grouped daily activity; turning it into a first-join cohort would require joining each actor's registration, onboarding and first membership timestamps with explicit deletion and late-event rules.

Parquet and the local object lake

What the batch wrote

```
s3://myscoutee-lake/daily_activity/
  day=2026-10-27/
    part-00000-...c000.snappy.parquet
  day=2026-10-26/
    part-00000-...c000.snappy.parquet
  _SUCCESS
```

Term	Meaning in this lab
Bucket / key	myscoutee-lake is the S3 bucket; daily_activity/day=.../part-...parquet is an object key. It is not a POSIX directory tree.
Parquet	Column-oriented file format with embedded schema. Spark can read selected columns and discover the day partition from the key path.
day partition	Spark's partitionBy('day') creates one key prefix per day; this is not a Kafka partition or ClickHouse MergeTree partition.
Snappy	Compression codec indicated by the part filename. The file contains grouped daily rows, not raw event JSON.
_SUCCESS	Spark's completion marker. The job also read the output back and counted 259 rows on this visit.

Inspect the live SeaweedFS volume

```
printf 'fs.ls /buckets/myscoutee-lake/daily_activity\n' | \
  docker compose -f stacks/core/compose.yaml exec -T lake \
  weed shell -filer localhost:8888
```

Look for myscoutee-lake, day=2026-09-28 through day=2026-10-27 on this visit, and _SUCCESS. A day prefix contains a .snappy.parquet object; a leftover _temporary prefix is staging metadata, not an additional final dataset. Open the SeaweedFS Filer UI at http://127.0.0.1:8888/buckets/myscoutee-lake/daily_activity/ to browse the same prefixes. It shows objects, not decoded Parquet rows. To exercise S3 authentication use an S3 client at http://127.0.0.1:8333 with lab-only credentials lab / lab-only-secret. No host AWS CLI is required for the shell exercise.

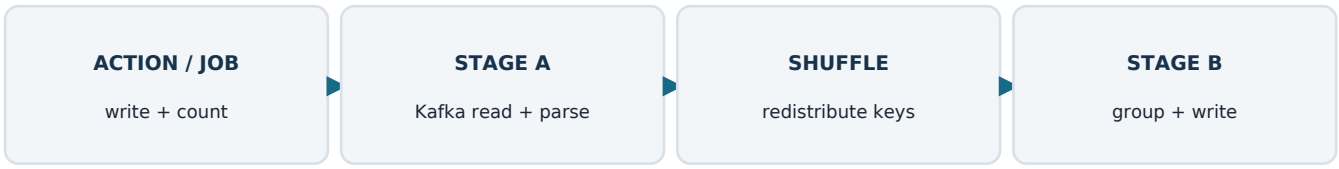
MyScoutee integration boundary

A future approved, minimized export could keep historical aggregates or cohort outputs in object storage for audited backfills. The existing Hive station does not query this bucket, and the dashboard reads ClickHouse rather than these Parquet files. SeaweedFS mini is one local node without a production durability design. Plan immutable output versions, catalog/schema ownership, access control, retention and deletion before using a lake for product data.

References: Spark Parquet and partition discovery; SeaweedFS mini S3 setup.

Spark execution and monitoring

From code to parallel work



Illustrative DAG: a shuffle can split stages; each stage runs many partition tasks.

Term	What to look for in this batch
Driver	spark_batch.py coordinates the application. local[*] runs driver and task work in one temporary container.
Cluster manager	Allocates executors in a real deployment (Standalone, YARN or Kubernetes). This lab uses local mode; no separate manager is running.
Job / action	A write or count action triggers execution. One application can have multiple jobs; do not assume one batch equals one job.
DAG / stage	Spark plans dependencies as a directed graph; shuffle boundaries usually divide work into stages. The figure is illustrative, not a literal UI plan.
Task / Spark partition	A stage launches tasks over partitions of its input. This is distinct from a Kafka topic partition and a Parquet day= prefix.

Open the completed run

```
./lab.sh batch
./lab.sh history-up
# Open http://127.0.0.1:18080/
./lab.sh history-down  # when finished
```

Select MyScoutee activity history. Jobs shows action-triggered runs and duration. Stages shows task counts, shuffle read/write and skew; SQL shows physical plans for DataFrame queries; Executors shows task and I/O metrics. A failed or slow task is examined at stage/task level, then checked against driver logs. The event log is stored locally in ignored build/spark-events; it does not contain source Kafka payloads. This UI is for execution monitoring, not MyScoutee product jobs or affinity state.

MyScoutee fit: use a reviewed, minimized export to rebuild historical participation or registration-to-first-join cohorts after logic changes. A slow stage or skewed partition would tell engineers where that backfill spends time; it would not tell an admin whether one user's registration or affinity recompute is correct. Product state remains in backend-owned flows; Airflow could schedule a future batch, while the History Server explains Spark's execution.

Reference: Spark 4.0.4 monitoring and History Server.

Parquet layout and analytical roles

Inside the files

A Parquet part file contains row groups. Within each row group, values are stored as column chunks, which are split into pages with encoding and compression. Metadata records the schema and statistics. A reader can select columns (column pruning), use predicates and statistics to skip some work (predicate pushdown), and read only relevant day= prefixes (partition pruning). These optimizations depend on the query, file layout and connector; a day prefix is outside the Parquet file, while row groups and pages are inside it.

Layer	Example in the lab	Why it matters
Object key	daily_activity/day=2026-10-27/part-....parquet	Select one date prefix for a narrower scan.
Parquet row group	A batch of grouped output rows inside a part file.	Readers may skip groups from metadata when filters permit.
Column chunk / page	events, actors or city values in encoded pages.	Read selected fields; Snappy reduces bytes on disk.

Roles and limits

Component	Best fit here	Current lab boundary
Spark	Recompute large historical datasets, joins and backfills.	One local batch over retained Kafka events.
ClickHouse	Fast repeated aggregate queries for an operational dashboard.	Reads its own ingested table, not this Parquet dataset.
SeaweedFS + Parquet	Store and exchange historical columnar output.	Single-node object store and overwritten prefix.
Airflow	Schedule and track dependencies between future exports and backfills.	The current DAG uses a separate fixture; no Spark launch.

A Parquet folder is a file-based lake dataset. It has no table-level snapshot or transaction metadata. Iceberg, Delta Lake or Hudi can add table semantics with a separate integration; the next page explains Iceberg using this lab's output. Spark SQL/DataFrames are exercised here; Structured Streaming, MLlib and GraphX are Spark capabilities, not parts of this run.

MyScoutee example: export approved registration, onboarding and first membership facts; define event time, deletion and late-arrival rules; compute one row per actor for a cohort, then write immutable dated output and reconcile it with backend counts. Partition by a useful query date only after measuring file sizes and read patterns. The existing daily_activity output is grouped event activity, so it cannot answer the person-linked cohort question by itself.

References: Apache Parquet file format; Spark Parquet and partition discovery; shared Spark and Parquet explanation reviewed for this section.

Spark + Parquet: table semantics

The actual application in this repository

The code lives in stacks/core/spark_batch.py and is versioned with the Compose launcher. ./lab.sh batch submits that Python application with spark-submit --master local[*]; the DataFrame operations in the script define the logic, while Compose passes packages, S3 settings, memory limits and event logging. One application can produce several Spark jobs when actions run. The History Server observes those jobs after completion; it is not a visual job editor. GitLab CI could test and package a future pipeline, and Airflow could schedule it, but neither currently launches this batch.

From a file set to a table

```
Spark SQL / compatible engine
  -> Iceberg catalog + current snapshot metadata
  -> referenced Parquet data files
  -> object storage (SeaweedFS S3 in this lab)
```

Capability	Current Parquet prefix	With an Iceberg table
Which files are current?	A reader lists the output prefix; overwrite replaces it.	Snapshot metadata names the committed file set.
Past state	The previous output is lost after this overwrite.	Time travel can read an older retained snapshot and its files.
Corrections / deletes	Require a custom rewrite and publication rule.	Table operations can commit changes atomically through a compatible engine.
Schema and partitions	Managed by each writer/reader contract and paths.	Table metadata supports controlled schema and partition evolution.

A snapshot is a consistent table version, not a Git commit and not a backup by itself. Time travel works only while the referenced metadata and data files remain available. A catalog identifies tables and their metadata; Parquet still holds the actual rows and SeaweedFS still stores the objects. Iceberg is a table format, not a separate compute engine or a replacement for Spark or ClickHouse.

MyScoutee decision

For an approved registration-to-first-join cohort, the current Spark job would first need actor-linked facts, event-time and deletion rules. If analysts later need repeatable corrections, concurrent readers and reproducible historical versions, an Iceberg table could make the published cohort safer to manage. The present lab has no Iceberg table, no Spark Iceberg connector and no configured catalog for this job; the output remains a simple overwritten Parquet prefix. Measure the need before adding that layer.

References: Spark application submission; Apache Iceberg concepts; Iceberg snapshots and time travel; shared Spark / Parquet / Iceberg discussion reviewed here.

Flink: a continuously running job

Follow the actual lab flow

```
synthetic affinity task transitions
  -> Kafka affinity.tasks.v1
  -> Flink SQL: 10-second TUMBLE by group + target
  -> Kafka affinity.rollups.v1
  -> ClickHouse affinity_rollups -> dashboard
```

Term	Meaning in this running example
SQL Client	Submits stacks/core/flink/affinity.sql to the session cluster, then exits; the job keeps running.
JobManager	Coordinates scheduling and checkpoints; its REST API and browser UI listen on localhost:8081.
TaskManager / slot	Executes operator subtasks. This lab has one TaskManager and two available task slots in total.
Source / operator / sink	Kafka task source; projection and window aggregation; Kafka rollup sink. The UI may chain several operators into one vertex.
Parallelism	How many subtasks run for an operator. The local SQL job uses default parallelism 1, not all Kafka partitions at once.
Window / state	A 10-second processing-time TUMBLE groups rows by group_id and target_type; state holds partial counts until a window closes.
Checkpoint	Periodic snapshot of processing state and source position for fault recovery. A successful checkpoint does not prove exactly-once delivery through every sink.

Processing time uses the clock when Flink sees a transition. A late or replayed record can land in a different window from its original run. Event-time windows would require an event timestamp and watermark strategy. A savepoint is an intentionally retained job-state snapshot for controlled stop, upgrade or migration; the current launcher does not create or restore one. The checkpoint volume persists across down, but a newly submitted lab job starts from earliest retained Kafka offsets rather than automatically resuming that old job.

Flink is the long-running computation here. Spark's batch recomputes a bounded historical snapshot and then exits; Airflow could schedule work but does not itself aggregate this stream. MyScoutee's backend still owns the durable recompute task, worker and affinity score; this Flink job only counts exported-style task transitions.

References: Flink deployment components; Flink checkpointing.

Flink: guided UI visit

1. Open the running cluster

```
./lab.sh stream-up
# Open http://127.0.0.1:8081/
```

The cluster overview shows one JobManager, one TaskManager, two slots and one running job on the documented visit. Select myscoutee-affinity-recompute-rollups. Its graph has a Kafka source/projection vertex and a window-aggregation/Kafka-sink vertex. Those boxes are execution units; their grouping can change with optimization and is not a business-process diagram.

UI area	What to inspect	How to interpret it
Job graph	Vertex status and record counters.	Green/RUNNING means operators are scheduled; counters show processed records, not distinct tasks.
Checkpoints	Completed, failed and in-progress snapshots; duration and size.	Look for repeated completion. A zero failure count is a local health sign, not an end-to-end guarantee.
Task Managers	One worker, slots and metrics.	An available slot is capacity; it is not a queued MyScoutee task.
Back Pressure / Metrics	Busy or blocked operators and throughput.	Investigate a slow source/sink or rising lag before drawing a performance conclusion.

2. Publish a small synthetic sample

```
./lab.sh affinity-seed --tasks 40 --seed 42
./lab.sh query "SELECT group_id, target_type, sum(pending),
  sum(done), sum(failed) FROM affinity_rollups FINAL
  GROUP BY group_id, target_type ORDER BY group_id, target_type"
```

This command publishes 40 synthetic task lifecycles, each with pending, running and done/failed transitions: 120 records. Wait for a 10-second window to close, then inspect the dashboard at http://127.0.0.1:8094. The Kafka topic and ClickHouse volume may contain earlier samples; absolute panel totals therefore need not equal 40. A restart from earliest retained Kafka data can replay transitions into new processing-time windows. The ClickHouse table merges duplicate window keys but cannot make this design an exact product counter.

MyScoutee use

After a reviewed backend-owned export, this kind of stream could report affinity recompute arrival rate, queue wait, completion and failure trends by target type or group. Reconcile every definition against AffinityRecomputeMetricsService before publication. It cannot calculate the affinity scores from these task-state signals, and the Mongo task queue remains the authoritative lifecycle. Stop the visit with ./lab.sh down; Docker volumes remain for later comparison.

Reference: Flink checkpoint UI; Flink dashboard REST API.

Flink: authoring and time semantics

Where the running job comes from

Flink is an execution engine, not a language or a visual flow editor. In this repository, stacks/core/flink/affinity.sql is the versioned job definition. ./lab.sh stream-up starts the session cluster, and the temporary flink-submit container runs SQL Client with -f against that file. CREATE TABLE declares the Kafka source and sink; INSERT INTO submits the continuous query. The SQL Client container exits while the job remains RUNNING in JobManager. The dashboard displays and monitors the job; its JAR upload screen is for a packaged application, not for editing this SQL.

Authoring route	How it is deployed	Used here?
Flink SQL	Write SQL in a file; run it with SQL Client or SQL Gateway.	Yes: affinity.sql via SQL Client.
DataStream / Table API	Develop an application, package a Java JAR, submit with CLI or the UI.	No JAR in this lab.
PyFlink	Write a Python application and submit it with Flink's Python CLI.	No Python Flink job here.

What each window means

Window	Task-transition example	Lab status
TUMBLE	One non-overlapping bucket per 10 seconds, group and target type.	Running; processing time.
HOP	A 10-second bucket every 5 seconds; a transition can contribute to overlapping buckets.	Concept only.
SESSION	Keep one burst together until a chosen inactivity gap closes it.	Concept only.

State, clocks and recovery

Window state holds partial counts until the window closes. Processing time uses Flink's clock when a Kafka record is processed; the current SQL declares PROCTIME() and no watermark. Event time would use an event timestamp from each record. A watermark expresses progress through event time and influences when windows close; a late transition needs an explicit lateness and correction policy. Do not interpret the current 10-second buckets as event-time history or exact task totals. Checkpoints periodically snapshot job state and source positions for recovery; savepoints are deliberate snapshots for controlled operations. Neither changes the at-least-once Kafka sink into an end-to-end exactly-once product counter.

MyScoutee decision

Both Flink and Spark can process bounded and unbounded data. This lab uses Flink for continuous affinity task-state rollups and Spark for a separate historical Parquet batch. A reviewed backend export could support per-target arrival, queue-wait and failure trends; the backend's recompute task and score stay authoritative. Profile views or affinity-score calculation are not emitted by this SQL. Add event-time handling, identity-safe keys, replay rules and reconciliation with AffinityRecomputeMetricsService before using these numbers beyond the sandbox.

References: Flink SQL Client; Flink window aggregation; Flink command line; shared Flink supplement reviewed for this section.

Avro and Schema Registry: different jobs

Why the extra contract layer?

Avro is a binary serialization format with a declared record schema. It describes field names and types and encodes event values compactly. A reader uses the writer schema to decode old bytes and may supply a newer reader schema to resolve added fields. Schema Registry is a separate service for storing schema versions and IDs and checking proposed changes against a compatibility policy. Kafka stores the bytes and routes records; it does not itself validate Avro or enforce a subject's evolution rules.

```
v1 Avro schema -> Registry subject -> schema ID n
record -> [magic byte][4-byte schema ID][Avro payload] -> Kafka
consumer -> ID lookup -> writer + v2 reader schema -> record
```

Part	Purpose	In this lab
Avro schema	Defines the event record and field types.	activity-v1.avsc and activity-v2.avsc.
Schema Registry	Stores subject versions/IDs and tests compatibility.	Optional service; REST on 127.0.0.1:8085.
Kafka	Retains partitioned event bytes for independent readers.	Separate activity.avro.v1 topic.
Serializer / reader	Writes Avro bytes, then uses the schema ID to decode.	schema_demo.py performs both operations.

A subject is the registry name for a schema history. This exercise uses myscoutee.activity.avro.v1-value, matching the Kafka topic and value side. The producer registers two versions under that subject. The record starts with Confluent's one-byte marker and four-byte schema ID; those bytes are transport framing, not part of the Avro schema. A generic Kafka console consumer sees binary data, not readable JSON.

Evolution shown in the exercise

Version 1 has event_id, occurred_at, event_key, group_id, city and mode. Version 2 adds source as a null-or-string field with default null. With BACKWARD_TRANSITIVE compatibility, a v2 reader can decode every registered earlier version; it supplies null when the v1 payload has no source. The demo checks that changing event_id from string to long is rejected. Registry checks schema compatibility, not business meaning, privacy, correct event ordering or consumer deployment.

The default JSON activity and affinity topics do not use this registry. The optional demo does not change the ClickHouse, Spark or Flink inputs. It starts only Kafka and the registry, avoiding the memory cost of the full core.

References: Apache Avro specification; Confluent schema evolution and compatibility.

Avro and Schema Registry: guided lab

1. Register and round-trip two versions

```
./lab.sh schema-demo
curl -s http://127.0.0.1:8085/subjects/\
myscoutee.activity.avro.v1-value/versions
./lab.sh down
```

The launcher starts the existing Kafka broker and an optional Schema Registry service, creates one separate topic, then runs a temporary Python producer/consumer. Its output names the two schema IDs, confirms that the incompatible field-type change was refused, and confirms both records decode with the v2 reader schema. The REST response lists two subject versions. Repeated runs reuse registered schemas and append new synthetic records; down preserves Kafka volumes. No registry UI, product credentials or backend data feed is present.

2. Interpret the result

Observation	What it demonstrates	What it does not prove
Two subject versions	One governed event contract can evolve.	Every existing consumer has been upgraded.
v1 reads as source=null	The v2 reader resolves an added defaulted field.	A change in event meaning is safe.
Breaking schema rejected	The policy blocks a string-to-long writer change.	All event contents are valid or authorized.
Two Kafka records	The bytes round-trip through the broker.	A production export is connected.

Where MyScoutee could use it

A reviewed backend-owned outbox could publish minimal post-commit facts such as event.member.joined or profile.onboarding to independently deployed analytics consumers. A stable event_id supports deduplication; group_id and approved dimensions support event-mode analysis. A person-linked cohort would additionally need an approved pseudonymous actor key and explicit retention and deletion rules. Avro would define the event envelope and value types; Registry would let teams test additions such as an optional source before publication. Kafka would fan out those approved facts. Keep raw identifiers and sensitive profile attributes out unless a documented purpose requires them.

Affinity task transitions are another possible contract: task_id, status, target type, group scope and timing fields could feed operational trends. The existing durable task service and MongoDB state would still own task execution and affinity scores. The current Redis admin telemetry and ClickHouse JSON path do not silently switch to Avro. An adoption plan would need backend outbox delivery, consumer migration, contract tests, access control, retention, deletion and reconciliation before any product use.

Use the schema first where several consumers share a long-lived event contract and independently deploy. For this small synthetic lab, Schema Registry is an educational extra service, not a measured production need.

References: Confluent Schema Registry; Confluent serializers and schema IDs.

Reading an Avro schema

The actual v2 contract

```
{
  "type": "record", "name": "ActivitySignal",
  "namespace": "myscoutee.lab",
  "fields": [
    {"name":"event_id", "type":"string"},
    {"name":"occurred_at", "type":"string"},
    {"name":"event_key", "type":"string"},
    {"name":"group_id", "type":"string"},
    {"name":"city", "type":"string"},
    {"name":"mode", "type":"string"},
    {"name":"source", "type":["null","string"],
     "default":null}
  ]
}
```

The .avsc file is JSON that describes data; a published event is a separate Avro binary record. type=record makes a named group of fields; name and namespace give that record an identity. Each entry in fields names one value and declares its type. string fields hold text; event_id is an event identity for deduplication, while event_key selects the synthetic lifecycle fact. The example keeps occurred_at as text for easy comparison with the existing JSON stream; a production contract could use an Avro timestamp logical type after agreeing on units and timezone.

Why the last field is written differently

source has a union type: null or string. The first union branch is null, and default null says what a v2 reader supplies for a v1 record that never carried source. This is why the existing v1 bytes still decode after v2 is introduced. The default is a reader-side resolution rule; it does not retroactively add bytes to old Kafka records. An incompatible required field or type change cannot be safely inferred from old data and is blocked by this lab's BACKWARD_TRANSITIVE policy.

Benefit	Concrete effect	Boundary
Typed contract	A serializer knows the expected field names and types.	Business rules still need application checks.
Compact record	Binary values omit repeated JSON field names.	Measure size and cost before choosing it.
Schema evolution	Reader and writer schemas resolve old and new records.	Compatibility policy must match deployment order.
Shared identity	The registry ID identifies the writer schema.	Registry availability and access need operations.

For MyScoutee, start with a minimal post-commit event rather than copying a whole profile document. Keep stable event identity, event time and only approved group or aggregate dimensions. Actor identity, deletion handling and retention need an explicit product data contract before any real export. The lab's city, mode and group values are synthetic examples, not permission to publish those attributes from production.

Reference: Apache Avro specification: schemas, unions, defaults and resolution.